

# خوارزميات التعلم الآلي

دليل شامل لخوارزميات SVM والغابة العشوائية وما بعدها

## Machine Learning Algorithms Explained

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

2026

عربي · English

# خوارزميات التعلم الآلي

*Machine Learning Algorithms Explained*

دليل شامل لخوارزميات SVM والغابة العشوائية وما بعدها

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

الطبعة الأولى — 2026 · First Edition

جميع الحقوق محفوظة للمؤلف

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

## إهداء

إلى كل باحث عربي يواجه بياناته بأدوات لم يتعلمها بلغته وإلى  
طلاب علم البيانات الذين يريدون الفهم قبل الحفظ هذا الكتاب  
يبدأ بالحدس وينتهي بالإتقان.

## تنبيه

هذا الكتاب أُنتج بمساعدة نموذج Claude Opus 4.7 من شركة Anthropic، تحت إشراف وتوجيه المؤلف. التوجيه الفكري واختيار الموضوع والمراجعة والمسؤولية النهائية للمؤلف الدكتور فضل محمد الأكوع. الكتاب لأغراض تثقيفية وإثرائية، ولا يُغني عن استشارة المتخصصين في الموضوعات القانونية والطبية والمالية والشرعية.

## **Notice**

This book was produced with the assistance of Anthropic's Claude Opus 4.7 model, under the author's direction and supervision. Intellectual direction, topic selection, review, and final responsibility rest with the author, Dr. Fadhl Mohammed Alakwaa. The book is for educational purposes and does not substitute for consultation with specialists in legal, medical, financial, or religious matters.

# المحتويات

---

- 1 الفصل الأول: ما هو التعلم الآلي؟
- 2 الفصل الثاني: الانحدار الخطي والانحدار اللوجستي
- 3 الفصل الثالث: شجرة القرار
- 4 الفصل الرابع: الغابة العشوائية
- 5 الفصل الخامس: آلة المتجهات الداعمة (SVM)
- 6 الفصل السادس: خوارزمية الجيران الأقرب (KNN)
- 7 الفصل السابع: خوارزميات التعزيز — XGBoost وال Gradient Boosting
- 8 الفصل الثامن: التجميع — K-Means و Hierarchical Clustering
- 9 الفصل التاسع: تقليل الأبعاد — PCA و t-SNE و UMAP
- 10 الفصل العاشر: التحقق من النموذج وتجنب الإفراط في التخصيص

11                      الفصل الحادي عشر: كيف تختار الخوارزمية المناسبة؟

---

12                      الفصل الثاني عشر: الخاتمة – التعلم الآلي والمستقبل

---



## الفصل الأول: ما هو التعلم الآلي؟

﴿وَعَلَّمَ آدَمَ الْأَسْمَاءَ كُلَّهَا﴾

— سورة البقرة: ٣١

في هذه الآية الكريمة إشارة عميقة إلى أن العلم يبدأ بالتسمية؛ أي بالتمييز بين الأشياء وإعطائها أسماء تدلّ على خصائصها. وهذا في جوهره ما يفعله التعلم الآلي: فهو لا يُلَقِّن الحاسوب قواعد جامدة، بل يعلِّمه أن يُسمّي الأشياء — أن يُصنّف الصورة كقطة أو كلب، أن يُعطي للورم اسم "حميد" أو "خيث"، أن يميّز بين البريد المهم والمزعج. التعلم الآلي هو فنّ تعليم الآلة كيف نتعرّف على الأنماط، تماماً كما علّم آدم — عليه السلام — أسماء الأشياء كلها.

### من البرمجة التقليدية إلى التعلم من الأنماط

في البرمجة التقليدية، يجلس المبرمج ويكتب قواعد صريحة: "إذا كانت درجة الحرارة فوق ٣٨ فالمرضى مصاب بحمى". هذه القواعد تعمل ما دامت المسألة بسيطة وواضحة. لكن ماذا لو طلبت منك أن تكتب قاعدة تميّز بين صورة قطة وصورة كلب؟ ستجد نفسك أمام جدار. كل قطة تختلف عن الأخرى في اللون والحجم والوضعية والإضاءة. ملايين القواعد لن تكفي.

هنا يدخل التعلم الآلي: بدلاً من كتابة القواعد، نعطى الحاسوب آلاف الأمثلة المُسمّاة — هذه قطعة، وهذه قطعة، وهذا كلب — ثم نتركه يستنبط القواعد بنفسه. الأنماط لا تُكتب، بل تُكتشف. هذه نقلة جوهرية في طريقة التفكير: من "كيف أحلّ هذه المشكلة؟" إلى "كيف أري الحاسوب أمثلة كافية ليتعلم بنفسه؟"

## الأنواع الثلاثة للتعلم الآلي

ينقسم التعلم الآلي إلى ثلاثة فروع رئيسية، وفهم الفرق بينها هو أول خطوة في رحلتك.

أولاً، التعلم الموجه (Supervised Learning). هنا نعطى الحاسوب بيانات مع إجاباتها الصحيحة. مثال: ألف صورة لمرضى مع تشخيصاتهم، أو ألف منزل مع أسعار بيعها. الحاسوب يتعلم العلاقة بين المدخلات والمخرجات، ثم يطبق ما تعلّمه على بيانات جديدة. معظم خوارزميات هذا الكتاب — الانحدار، الغابة العشوائية، آلة المتجهات الداعمة — تنتمي إلى هذا الفرع.

ثانياً، التعلم غير الموجه (Unsupervised Learning). هنا نعطى الحاسوب بيانات بلا إجابات، ونطلب منه أن يجد البنية الخفية فيها. كأن نقول له: "هذه عشرة آلاف خلية من خزعة سرطانية، اعر على المجموعات التي تتشابه فيما بينها." خوارزميات التجميع وتقليل الأبعاد — مثل K-Means و PCA — هي أبطال هذا الفرع.

ثالثاً، التعلم بالتعزيز (Reinforcement Learning). وهو الأقرب إلى طريقة تعلّم الإنسان: الحاسوب يُجرب ويتعلم من المكافآت والعقوبات. هكذا تعلّم برنامج AlphaGo لعب الشطرنج وتفوّق على البشر. هذا النوع لن نعمّق فيه في هذا الكتاب، لكنه جوهر الكثير من التطبيقات الحديثة.

## خط الإنتاج: من البيانات إلى التنبؤ

كل مشروع تعلم آلي يمر بخمس مراحل أساسية، وفهمها يُجنّبك كثيراً من الأخطاء لاحقاً.

تبدأ المرحلة الأولى بالبيانات: جمعها وتنظيفها والتأكد من جودتها. كثير من الباحثين يظنون أن السرّ في الخوارزمية، لكن الحقيقة أن ثمانين بالمئة من نجاح أي مشروع يكمن في جودة البيانات. بيانات سيئة لن تُنتج نموذجاً جيداً مهما كانت الخوارزمية ذكية.

ثم تأتي هندسة الميزات (Feature Engineering): تحويل البيانات الخام إلى أرقام ذات معنى يفهمها النموذج. مثلاً، تاريخ الميلاد لا يفيد النموذج مباشرة، لكن "العمر بالسنوات" يفيده. هذه المرحلة تتطلب فهماً عميقاً للمجال – الطبي، المالي، أو غيره.

بعدها يأتي النموذج: اختيار الخوارزمية المناسبة وتدريبها على البيانات. ثم التنبؤ: استخدام النموذج المدرب على بيانات جديدة لم يرها من قبل. وأخيراً التقييم:

قياس مدى دقة النموذج بمقاييس واضحة، وهذا ما سنتعمق فيه في الفصل العاشر.

## لماذا تبقى الخوارزميات الكلاسيكية مهمة؟

في عصر نتصدر فيه الشبكات العصبية العميقة العناوين، قد يتساءل القارئ: لماذا نتعلم الغابة العشوائية أو الانحدار اللوجستي أصلاً؟ والجواب من خبرتي العملية: لأنها — في معظم المشاكل الحقيقية — أفضل خيارائك.

التعلم العميق يحتاج إلى ملايين الأمثلة وأجهزة قوية. أما في الأبحاث الطبية، فقد تجد نفسك مع مئتي مريض فقط. هنا تتفوق الخوارزميات الكلاسيكية بفارق واضح. كذلك تمتاز هذه الخوارزميات بقابلية التفسير: يمكن للطبيب أن يسألك "لماذا توقع نموذجك أن هذا المريض في خطر؟" وأن تُجيبه بإجابة واضحة. هذا أمر عسير في الشبكات العميقة.

## نظرة من المختبر

في مختبري بجامعة ميشيغان، نعمل على بيانات الجينوم وحيد الخلية (Single-Cell RNA-seq) لفهم أمراض كالسرطان وأمراض الكبد الدهني. هذه البيانات معقدة: قد تحتوي العينة الواحدة على ثلاثين ألف جين عبر عشرات الآلاف من الخلايا. لإيجاد الخلايا الشاذة أو تمييز أنواع الخلايا، نعتمد يومياً على

الغابة العشوائية، UMAP، K-Means، وغيرها من الخوارزميات التي سترها في هذا الكتاب.

لقد رأيت بأمّ عيني نموذج Random Forest يكتشف بصمات بيولوجية لم نكن نعرفها، ورأيت SVM تُصنّف خلايا سرطانية بدقة تفوق التقنيات اليدوية القديمة. هذه ليست أدوات نظرية، بل أدوات تُستخدم في أبحاث تُغيّر حياة المرضى.

## ما الذي ستتعلمه؟

في الفصول القادمة، ستتعرف على عشر خوارزميات رئيسية، كلٌّ منها بتشبيه ومثاله الواقعي. لن أغرقك بالرياضيات في البداية — سنبدأ دائماً بالحدس. وحين تأتي المعادلات، ستظهر في صناديق جانبية اختيارية لمن أراد التعمق. الهدف أن تخرج من هذا الكتاب بفهم عميق، لا بحفظ معادلات.

صندوق للمتعمقين: عندما نقول إن "النموذج يتعلم"، نعي رياضياً أنه يجد قيمًا للمعاملات (*parameters*) تُقلّل دالة الخطأ (*loss function*) بين تنبؤاته والقيم الحقيقية. كل خوارزمية تختلف في شكل الدالة وطريقة تقليلها، لكن المبدأ واحد.

تذكر دائماً: التعلم الآلي ليس سحراً، بل هو فنّ تنظيم البيانات وانتقاء الأدوات المناسبة. ابدأ بالحدس، ثم ابنِ عليه.

## الفصل الثاني: الانحدار الخطي والانحدار اللوجستي

﴿إِنَّ اللَّهَ يَأْمُرُ بِالْعَدْلِ وَالْإِحْسَانِ﴾

— سورة النحل: ٩٠

العدل — في معناه العميق — هو وضع كل شيء في موضعه الصحيح، وإيجاد العلاقة المتوازنة بين السبب والنتيجة. والانحدار الخطي، في جوهره، عدل رياضي: نبحث عن الخط الذي يعدل بين النقاط، يقترب منها قدر الإمكان دون أن يتطرق نحو أيٍّ منها. كل خطوة في تعلّم الآلة هي محاولة لاكتشاف العلاقة العادلة بين المتغيرات.

### الانحدار الخطي: حين تُريد أن تتنبأ برقم

تخيّل أنك تبيع بيتاً، وتريد أن تعرف سعره العادل. تنظر إلى عشرات البيوت المماثلة في حيّك التي بيعت في السنة الماضية. تجد أن المساحة الأكبر تُقابل سعراً أعلى، عموماً. لو رسمت نقاطاً على ورقة — كل نقطة تمثّل بيتاً، الأفقي مساحة، والرأسي سعر — لرأيت سخابة من النقاط تميل صعوداً.

سؤال الانحدار الخطي بسيط: ما هو أفضل خط مستقيم يمر وسط هذه السحابة؟ خط لو رسمناه، لكان أقرب ما يكون إلى جميع النقاط معاً. هذا الخط ليس مجرد

حساب جمالي، بل هو معادلة تنبؤ: أعطني مساحة بيت جديد، وسأخبرك بسعره المتوقع.

التشبيه الأشهر هنا هو خط أفضل ملائمة (Line of Best Fit). الفكرة أن النموذج يحاول أن "يعدل" بين كل نقطة، فلا يقترب جداً من بعضها على حساب الأخرى. كيف يقيس النموذج هذا العدل؟ يحسب مجموع مربعات المسافات بين كل نقطة والخط، ويختار الخط الذي يجعل هذا المجموع أصغر ما يمكن. هذا ما يُسمى رياضياً المربعات الصغرى.

## ماذا يعني "خطي"؟

كلمة "خطي" مهمة. تعني أن العلاقة بين المُدخل والمُخرج مستقيمة: زيادة في المُدخل بمقدار وحدة تُقابل زيادة ثابتة في المُخرج. لو كانت العلاقة منحنية — مثلاً السعر يقفز بشكل غير متوقع بعد مساحة معينة — فالانحدار الخطي البسيط لن يلتقطها بدقة. لكنه يبقى خطأً أساسياً يُريك الاتجاه العام بسرعة، وغالباً ما يكون نقطة البداية في أي تحليل.

صندوق للمعمّقين: المعادلة الأساسية للانحدار الخطي هي:  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$  حيث  $\beta$  هي المعاملات التي يتعلّمها النموذج، و  $\varepsilon$  هو الخطأ غير المُفسّر.



## الانحدار اللوجستي: حين يكون السؤال نعم أو لا

لكن ماذا لو لم نكن نتنبأ برقم، بل بفئة؟ هل سيصاب هذا المريض بالسكري نعم أم لا؟ هل ستؤدي هذه الجرعة من الدواء إلى استجابة أم لا؟ هنا لا يصلح الانحدار الخطي، لأن الإجابة ليست رقماً مفتوحاً، بل اختيار من خيارين.

يدخل الانحدار اللوجستي إلى المسرح. اسمه يحوي كلمة "انحدار" لكنه في الحقيقة خوارزمية تصنيف. هو يأخذ المدخلات ذاتها، ويحسب لها رقماً، لكنه يمرر هذا الرقم عبر دالة سحرية تُسمى الدالة السينية (Sigmoid) التي تضغط أي رقم — مهما كان كبيراً أو صغيراً — إلى قيمة بين الصفر والواحد. هذه القيمة هي احتمال الانتماء إلى الفئة الموجبة.

تخيّل الدالة السينية كأنها منحنى على شكل حرف S ممدود. عند القيم السالبة الكبيرة، يقترب الناتج من الصفر؛ عند القيم الموجبة الكبيرة، يقترب من الواحد؛ وعند الصفر، يكون الناتج بالضبط 0.5. هذا الانحناء اللطيف هو سرّ الخوارزمية: فهو يُحوّل خطأً مستقيماً (انحداراً عادياً) إلى مُصنّف احتمالي.

## مثال طبي: التنبؤ بخطر السكري

في دراسة افتراضية على مرضى يحملون عوامل خطر السكري، نجّج لكل مريض ثمانية متغيرات: العمر، مؤشر كتلة الجسم، ضغط الدم، مستوى السكر الصائم،

عدد مرات الحمل (للنساء)، إنخ. ولكل مريض نعرف هل أُصيب لاحقاً بالسكري أم لا.

نُدرب نموذج انحدار لوجستي. النموذج يتعلم أن كل متغير يُعطى معاملًا يدلّ على أهميته. مثلاً، السكر الصائم قد يحصل على معامل عالٍ موجب: كلما ارتفع، زاد احتمال الإصابة. ضغط الدم أقل أهمية لكنه ما يزال يُساهم. بعض المتغيرات قد تحصل على معاملات قريبة من الصفر، ما يعني أنها لا تُساعد.

عندما يأتي مريض جديد، ندخل قياساته في المعادلة، نحصل على رقم، نمُرّه عبر الدالة السينية، ونحصل على احتمال — مثلاً ٠,٧٢ — يعني أن هناك ٧٢٪ احتمال إصابته. إذا تجاوز هذا الاحتمال عتبة محددة (غالباً ٠,٥)، نُصنّفه "في خطر".

## متى تختار أيّاً منهما؟

القاعدة بسيطة: إذا كنت تتنبأ برقم مستمر — سعر، طول، تركيز دواء في الدم — فأنت تحتاج انحداراً خطياً. إذا كنت تتنبأ بفئة — مريض/سليم، يشتري/لا يشتري، ناجح/راسب — فأنت تحتاج انحداراً لوجستياً.

كلتا الخوارزميتين بسيطتان وسريعتان وقابلتان للتفسير. وهذا الأخير ميزة عظيمة في المجال الطبي. حين تخبر طبيباً أن "كل زيادة سنة في العمر تزيد لوغاريتم

الاحتمالات بمقدار ٠.٠٤، فأنت تُعطيهِ أداة قابلة للنقاش والمراجعة، لا صندوقاً أسود.

## الأخطاء الشائعة

من أكثر الأخطاء شيوعاً: استخدام الانحدار الخطي على بيانات غير خطية بطبيعتها. لو كانت العلاقة بين الجرعة والاستجابة منحنية على شكل U، فإن انخط المستقيم سيُخطئ في الطرفين. الحل أحياناً تحويل المتغيرات (لوغاريتمياً مثلاً) أو الانتقال إلى نموذج أكثر مرونة.

خطأ آخر: تجاهل التعدد الخطي (Multicollinearity)، حين تكون متغيراتك مرتبطة فيما بينها. مثلاً وزن الجسم وكثافة الجسم. يصبح النموذج غير مستقر، والمعاملات صعبة التفسير. الحل عادة إزالة أحد المتغيرين أو دمجها.

## نهاية الفصل

الانحدار الخطي واللوجستي ليسا الأكثر إبهاراً، لكنهما الأساس الذي تُبنى عليه كل خوارزميات التصنيف الحديثة. تعلّهما جيداً، ولن يخيب الانحدار اللوجستي ظنك في كثير من المشاكل الواقعية. أحياناً، أبسط أداة هي الأمثل.

## الفصل الثالث: شجرة القرار

﴿أَلَمْ تَرَ كَيْفَ ضَرَبَ اللَّهُ مَثَلًا كَلِمَةً طَيِّبَةً كَشَجَرَةٍ طَيِّبَةٍ﴾

— سورة إبراهيم: ٢٤

ضرب الله المثل بالشجرة لأنها تُجسّد فكرة عميقة: أصلٌ واحد ينبثق منه فروع كثيرة، كل فرع يُؤدّي إلى ثمرٍ مختلف. وهذا بالضبط ما تفعله شجرة القرار في التعلم الآلي: تبدأ من سؤال واحد في الجذر، ثم تتفرّع إلى أسئلة فرعية، حتى تصل أوراقها إلى الإجابات النهائية. الشجرة الطيبة تُعطي ثمرًا طيبًا، والشجرة المدربة جيدًا تُعطي تنبؤات دقيقة.

### التشبيه الأقرب: مخطط انسيابي يُجيب عنك

تخيّل أنك في عيادة طبيب باطنية يستقبل مريضاً يشكو من ألم في الصدر. الطبيب لا يطلب كل الفحوصات دفعة واحدة، بل يسأل أسئلة متتابعة. السؤال الأول: هل المريض فوق الستين؟ إذا نعم، السؤال الثاني: هل يدخن؟ إذا نعم، السؤال الثالث: هل ضغطه مرتفع؟ بناءً على الإجابات، يصل الطبيب إلى قرار: نوجّهه لإجراء تخطيط قلب، أو يُعطيه دواءً للحموضة، أو يطلب اختباراً للبروتين العضلي.

هذا بالضبط هو شكل شجرة القرار. سلسلة أسئلة "نعم/لا" نتفرّع، وكل ورقة في النهاية تُمثّل قراراً. الجمل في هذا التشبيه أنه ليس تشبيهاً فقط — بل هو حرفياً ما تبنيه الخوارزمية.

## التشريح: عقد، فروع، وأوراق

تكوّن شجرة القرار من ثلاثة عناصر:

**العقدة (Node):** نقطة فيها سؤال، مثل "هل العمر فوق ٥٠؟". نتفرّع منها فروع.

**الفرع (Branch):** مسار يؤدي إلى عقدة أخرى أو إلى ورقة، حسب إجابة السؤال.

**الورقة (Leaf):** نهاية المسار، تحوي القرار النهائي — الفئة المتوقعة أو القيمة المتوقعة.

العقدة الأولى تُسمى الجذر (Root). كلما نزلت في الشجرة، ضاقت البيانات، حتى تصل إلى ورقة فيها مجموعة صغيرة من العينات المتشابهة جداً.

## كيف تختار الشجرة سؤالها التالي؟

السؤال الأذكى الذي قد يخطر ببالك هنا: من الذي يُقرّر ما هو السؤال الأول في الجذر، ثم الثاني، وهكذا؟ الإجابة: الخوارزمية نفسها، بطريقة منهجية.

في كل عقدة، ننظر الخوارزمية إلى كل المتغيرات وكل القيم المحتملة، وتُسأل: أيّ سؤال يُفصل البيانات أكثر؟ أيّ سؤال يُنتج مجموعتين فرعيتين أكثر "نقاءً"؟

كلمة "نقاء" مهمة. لو كانت لدينا ١٠٠ مريض، ٥٠ مصاب و ٥٠ سليم، فهذا أعلى درجات "عدم النقاء". لكن لو سأل النموذج "هل السكر فوق ١٢٠؟" وانقسم المرضى إلى مجموعتين، الأولى ٤٠ مصاب و ٥ سليم، والثانية ١٠ مصاب و ٤٥ سليم — فقد تحقّق نقاء كبير. كل مجموعة أصبحت تميل لفئة واحدة.

تقيس الخوارزمية هذا النقاء بمقاييس مثل الإنتروبيا (Entropy) أو مؤشر جيني (Gini). الفرق بينهما تقني، لكن الفكرة مشتركة: انتقاء السؤال الذي يُقلّل الفوضى بأقصى ما يمكن. هذا ما يُسمى مكسب المعلومات (Information Gain).

صندوق للمتعمقين: الإنتروبيا لمجموعة بها فئتان بنسبتي  $p$  و  $(p-1)$   
 تُحسب ك:  $H = -p \log_2(p) - (1-p) \log_2(1-p)$ . تكون أعلى قيمة عند  $p = 0.5$  (فوضى تامة) وأدناها عند  $p = 0$  أو  $1$  (نقاء تام).

## مشكلة الإفراط في التخصيص

الشجرة الجشعة تريد أن تفصل البيانات تماماً، وتستمر في التفرع حتى يصبح في كل ورقة عينة واحدة فقط. هذا يُعطي دقة مثالية على بيانات التدريب، لكنه نغّ مدّمّر على بيانات جديدة.

السبب أن الشجرة "حفظت" التدريب بدلاً من أن "تتعلم" منه. صارت تلتقط ضوضاء وعشوائية لا تتكرر. هذا يُسمى الإفراط في التخصيص (Overfitting)، وهو من أهم المشاكل في التعلم الآلي عموماً.

## التشذيب: حلّ مشكلة الشجرة المتعجرفة

الحلّ يأتي بالتشذيب (Pruning). تماماً كما يُشَدَّب البستاني الفروع الزائدة لتنمو الشجرة بصحة، نُشَدَّب الفروع التي لا تُضيف قيمة حقيقية.

هناك طريقتان للتشذيب: التشذيب القبلي (Pre-pruning) يضع حدوداً قبل البناء، مثل: لا تتجاوز عمقاً معيناً، أو لا تُفرّع إن كان عدد العيّنات أقل من ٢٠. أما التشذيب البعدي (Post-pruning) فيبني الشجرة كاملة ثم يحذف الفروع التي تُسبب إفراطاً.

النتيجة شجرة أقل دقة على التدريب، لكن أكثر دقة على البيانات الجديدة – وهذا هو المهم.

## مثال عيادي: قرار الإحالة لفحص إضافي

لنتخيّل عيادة أولية تستقبل مرضى يشكون من تعب وألم بطن. الطبيب يُريد نظاماً يُساعده في تحديد من يُحال إلى متخصص.

ندرب شجرة قرار على ٢٠٠٠ سجل سابق. نتعلّم الشجرة سلسلة الأسئلة: هل وزن المريض هابط بشكل غير مُفسّر؟ إذا نعم، هل هناك دم في البراز؟ إذا نعم، يُحال فوراً. إذا لا، هل قيم خميرة الكبد مرتفعة؟ إذا نعم، فحص إضافي. وهكذا.

الجمال هنا أن الطبيب يستطيع أن يقرأ الشجرة ويفهم منطق كل قرار. هذا ما يُسمى التفسيرية (Interpretability)، وهي ميزة شجرة القرار الأبرز.



## متى لا تستخدم شجرة القرار؟

رغم بساطتها وجاذبيتها، لا تُناسب شجرة القرار كل المسائل. عندما تكون البيانات كثيرة الميزات والعلاقات بينها معقدة، تكون شجرة واحدة هشة جداً وتقع في الإفراط بسهولة. كذلك تكون غير مستقرة: تغيير بسيط في البيانات قد يُغيّر شكل الشجرة كلياً.

الحل لهذه المشاكل ليس التخلي عن الفكرة، بل الانتقال إلى الغابة العشوائية — موضوع الفصل القادم — التي تجمع مئات الأشجار في حشد واحد لتُعطي قراراً أقوى.

## الخلاصة

شجرة القرار هي نقطة الانطلاق المثالية لفهم التصنيف، لأنها تعكس طريقة تفكيرنا البشرية: سلسلة أسئلة تنتهي بقرار. تعلّمها بعمق، فهي اللبنة الأساسية لكثير من الخوارزميات المتقدمة التي سترها لاحقاً، من الغابة العشوائية إلى

.XGBoost

## الفصل الرابع: الغابة العشوائية

﴿وَشَاوِرْهُمْ فِي الْأَمْرِ﴾

— سورة آل عمران: ١٥٩

أمر الله نبيه ﷺ بالشورى رغم أنه يُوحى إليه، إعلاءً لقيمة الرأي الجماعي. فالعقول كلها اجتمعت، انكشفت زوايا لم يرها العقل الواحد. وهذا بالضبط جوهر الغابة العشوائية: لا نسأل شجرة قرار واحدة، بل نسأل مئات الأشجار، ثم نُجمع آراءها في قرار واحد. إنها شورى رياضية بامتياز.

### حكمة الجماعة: من خير واحد إلى لجنة خبراء

تخيّل أنك تريد تشخيصاً طبياً مهماً. هل ستكتفي برأي طبيب واحد؟ غالباً ستطلب رأياً ثانياً، ثم ثالثاً، خاصة إذا كان القرار خطيراً. لماذا؟ لأن كل طبيب يحمل تجارب وخبرات مختلفة، وكل واحد قد يلاحظ ما فاتته الآخر. حين يتفق ثلاثة أطباء على تشخيص، تكون ثقتك أكبر بكثير من ثقتك في طبيب واحد.

هذا هو منطق الغابة العشوائية. شجرة قرار واحدة ذكية، لكنها قد تخطئ. خطأها في حالة واحدة قد يقع، لكن في معظم الحالات تُصيب. السرّ هو أن نسأل

١٠٠ شجرة، كل واحدة درّبت بطريقة مختلفة قليلاً، ثم نأخذ التصويت الجماعي. الأخطاء الفردية تتقاطع وتُلغى بعضها، بينما الإجابة الصحيحة تتعزّز.

## كيف نبني هذه الأشجار؟

السؤال المهم: كيف نضمن أن كل شجرة في الغابة تختلف عن أخواتها؟ لو درّبنا ١٠٠ شجرة على نفس البيانات بنفس الطريقة، لكنت كلها متطابقة، ولا يكون هناك فائدة من الجماعة. الحلّ هو إدخال العشوائية بطريقتين ذكيتين.

العشوائية الأولى: عيّنات البيانات (Bagging). لكل شجرة، نأخذ عيّنة عشوائية من البيانات الأصلية مع التكرار. لو كانت لدينا ١٠٠٠ مريض، نسحب ١٠٠٠ سجل عشوائياً، حيث قد يتكرّر بعض المرضى ويُهمل البعض الآخر. هكذا تكون كل شجرة قد رأت بيانات مختلفة قليلاً.

العشوائية الثانية: الميزات (Feature Sampling). عند كل عقدة في الشجرة، بدلاً من النظر إلى كل المتغيرات، نختار عشوائياً مجموعة فرعية منها — مثلاً ٥ من أصل ٢٠. هذا يُجبر كل شجرة على بناء قراراتها بمنظور مختلف. شجرة قد تركز على عوامل العمر والوزن، وأخرى على الفحوصات المخبرية.

النتيجة: مئات الأشجار، كلٌّ منها مُخصصة قليلاً ومخطّئة بطريقتها الخاصة. وحين تتجمع، تتلاقى على الإجابة الصحيحة.

## التصويت: كيف نُجمع الآراء؟

في مسألة تصنيف (مثل: هل سيُصاب المريض بالسكري؟)، كل شجرة تُعطي رأياً (نعم أو لا)، والغابة تأخذ الأغلبية. لو قالت ٧٢ شجرة "نعم" و ٢٨ "لا"، فالقرار الجماعي "نعم".

في مسألة انحدار (مثل: ما هو سعر هذا البيت؟)، كل شجرة تُعطي رقماً، والغابة تأخذ المتوسط. هذه البساطة في التجميع هي قوة الخوارزمية.

صندوق للمتعمقين: نظرياً، الجمع بين عدة نماذج "ضعيفة لكن مستقلة" يُقلّل التباين (*Variance*) للنموذج النهائي بشكل ملحوظ، وفقاً لقاعدة توفر الاستقلالية. كلما كانت الأشجار أقلّ ارتباطاً ببعضها، كان الأداء أفضل.

## لماذا الغابة العشوائية تفوز دائماً تقريباً؟

في تجاربي، كثيراً ما تكون الغابة العشوائية أول ما أجربه على أيّ بيانات جديدة. لماذا؟ لأنها:

نتعامل بسلاسة مع البيانات الكثيرة الأبعاد. لو كان لديك ٥٠٠٠ متغير، الغابة العشوائية لا تُربك. وتعمل جيداً مع البيانات الناقصة (Missing Data) دون تحضير معقد. كذلك لا تتأثر بشدة إذا لم تكن البيانات منظمّة بصيغة موحّدة (Standardization)، بعكس بعض الخوارزميات التي تتأثر فعلاً.

والأهم: تعطيك أهمية الميزات (Feature Importance). بعد التدريب، تستطيع الغابة أن تخبرك أيّ المتغيرات كانت الأكثر استخداماً في القرارات. هذا كنز تشخيصي. في الأبحاث الطبية، نستخدم هذه الأهمية لاكتشاف بصمات حيوية لم يكن أحد يتوقعها.

## تجربتي مع بيانات الجينوم وحيد الخلية

في مختبري بجامعة ميشيغان، نعمل على بيانات الجينوم وحيد الخلية لسرطان الكبد. كل عيّنة تحتوي على ٢٥٠٠٠ جين عبر ٣٠٠٠٠ خلية. الهدف: تمييز الخلايا السرطانية عن الخلايا الطبيعية.

استخدمنا غابة عشوائية بألف شجرة. النتيجة كانت دقة تتجاوز ٩٤٪. الأهم أن الغابة كشفت لنا قائمة بأهم ٥٠ جيناً تساهم في القرار. تمت دراسة هذه الجينات بيولوجياً، وتبين أن بعضها له دور حقيقي في تطوّر السرطان لم يكن مُعلنًا. أداة بسيطة، نتيجة مذهلة.

## متى لا تكون الغابة العشوائية الخيار الأفضل؟

لكلّ خوارزمية حدودها. الغابة العشوائية بطيئة نسبياً عند التنبؤ مقارنةً بالحداد لوجستي بسيط، لأنها تستشير مئات الأشجار. لو كنت تحتاج تنبؤاً في أجزاء من الثانية على ملايين الطلبات، قد تكون أبطأ من اللازم.

كذلك تفقد قابلية التفسير المباشرة الموجودة في شجرة واحدة. لا يمكنك "قراءة" غابة من ١٠٠ شجرة كما تقرأ شجرة واحدة. وتأتي مفاضلة بين الدقة العالية والتفسير الواضح.

## مفهوم ال Out-of-Bag

من خصائص الغابة العشوائية الجميلة: عند بناء كل شجرة، حوالي ثلث البيانات لا تُستخدم في تدريبها (لأن العينة كانت عشوائية). يمكن استخدام هذه البيانات المتروكة لتقييم الشجرة دون الحاجة إلى تقسيم تدريب/اختبار خارجي. هذا ما يُسمى خطأ Out-of-Bag، وهو تقدير مجاني لأداء النموذج.

## الخلاصة

الغابة العشوائية أحد أعظم اختراعات التعلم الآلي العملي. سهلة الاستخدام، قوية، ومرنة. ابدأ بها دائماً قبل أن تجرب خوارزميات أعقد. إذا كانت إجابتها

كافية، فلا داعي للذهاب أبعد. الشورى – كما علمنا قرآنا – كثيراً ما تُغني عن  
البحث عن خير وحيد.

## الفصل الخامس: آلة المتجهات الداعمة (SVM)

﴿وَجَعَلْنَا بَيْنَهُمْ وَبَيْنَ الْقُرَى الَّتِي بَارَكْنَا فِيهَا قُرًى ظَاهِرَةً﴾

— سورة سبأ: ١٨

في هذه الآية إشارة إلى وجود فواصل واضحة بين الأماكن، فواصل تُسهّل العبور وتُنظّم الحدود. وآلة المتجهات الداعمة — أو SVM — تقوم على فكرة أصيلة شديدة الشبه: كيف نرسم الفاصل الأكثر وضوحاً، الأكثر بُعداً عن كل طرف، حتى يكون التمييز سهلاً ولا يتداخل التصنيف. هي خوارزمية تبحث عن "الطريق الأعرض" بين فئتين.

### التشبيه الأساسي: الطريق الأعرض

تخيل أن لديك مجموعة من النقاط الزرقاء على ورقة، وأخرى من النقاط الحمراء. مطلوب منك أن ترسم خطاً يفصل بينهما. كم خطأ ممكناً أن ترسمه؟ مئات. لكن أيها الأفضل؟

SVM يقول: ارسم الخط الذي يكون بمنزلة منتصف طريق عريض بين الفئتين. كلما كان الطريق أعرض، كان الفصل أوضح. النقاط الزرقاء تبقى على رصيف،



والخبراء على رصيف آخر، والمسافة بينهما أكبر ما يمكن. هذا الطريق يُسمى الهامش (Margin).

النقاط التي تقع على حافتي الطريق - الأقرب إلى الخط الفاصل - هي متجهات الدعم (Support Vectors). هي وحدها التي تحدّد موقع الخط. لو حذفنا كل النقاط الأخرى، لن يتغيّر شيء. هذه فكرة عميقة: الخوارزمية تعتمد على القليل من النقاط الحرجة، وهذا ما يجعلها فعّالة في البيانات الصغيرة.

## ما هو "المستوى الفائق" دون رياضيات معقّدة؟

في عالم ثنائي الأبعاد (الورقة)، الفاصل خط. في ثلاثة أبعاد، الفاصل سطح مستوٍ. وفي أبعاد أعلى - قد يكون لدينا ٢٠ متغيّراً - الفاصل شيء يصعب تخيّلُه، يُسمى مستوى فائق (Hyperplane).

لا تخف من الكلمة. ببساطة: هو نسخة "ذات أبعاد كثيرة" من الخط المستقيم. الفكرة الجوهرية لا تتغيّر: نريد فصل الفئات بأقصى هامش ممكن في فضاء البيانات، مهما كان عدد أبعاده.

## الحيلة العبقريّة: حين لا تكون البيانات قابلة للفصل خطياً

ماذا لو لم تكن البيانات تنفصل بخط مستقيم أصلاً؟ تخيّل أن النقاط الحمراء تشكّل دائرة في الوسط، والزرقاء تحيط بها. لا يوجد خط مستقيم يفصلها.

هنا تأتي حيلة النواة (Kernel Trick)، وهي إحدى أجمل الأفكار في التعلم الآلي. التشبيه: تخيّل أن لديك ورقة عليها نقاط لا يمكن فصلها بخط. ماذا لو طويت الورقة بطريقة معيّنة، أو رفعت بعض النقاط لأعلى لتصبح في فضاء ثلاثي الأبعاد؟ في الفضاء الجديد، قد تجد سطحاً مستوياً يفصلها بسهولة.

ال Kernel هو دالة رياضية تنقل البيانات إلى فضاء أعلى أبعاداً — أحياناً لانهائي — دون أن تحتاج فعلياً إلى حساب الإحداثيات الجديدة. النواع الشائعة:

النواة الخطية (Linear Kernel) — لا تحويل، تستخدم حين تكون البيانات بالفعل قابلة للفصل خطياً.

النواة متعددة الحدود (Polynomial Kernel) — تُحوّل البيانات بعلاقات تربيعية أو تكعيبية.

النواة الغاوسية (RBF Kernel) — الأكثر شيوعاً. نتعامل مع علاقات معقّدة عبر دالة جرس مركزية. هي خيارك الافتراضي إذا لم تكن متأكداً.

صندوق للمتعمقين: نواة RBF تأخذ شكل  $K(x, y) = \exp(-\gamma \|x - y\|^2)$ .  
المعامل  $\gamma$  يُحدّد "سرعة" انخفاض التأثير مع المسافة. قيم صغيرة لـ  $\gamma$  تُعطي قراراً ناعماً، وقيم كبيرة تُعطي قراراً ينجني بقوة.

## الهامش الصلب مقابل الهامش اللين

في العالم المثالي، توجد فجوة واضحة بين الفئات. لكن في العالم الحقيقي، البيانات فوضوية: قد تكون هناك نقاط زرقاء تتسلّل بين الحمراء، أو أخطاء في التسمية. هل نُجبر النموذج على فصل تام، حتى لو شوّه ذلك الحدود؟

الهامش الصلب (Hard Margin) يُصرّ على الفصل التام. هذا غير واقعي في معظم الحالات.

الهامش اللين (Soft Margin) يسمح بعدد محدود من الأخطاء — نقاط تخترق الهامش أو تُصنّف خطأً — مقابل الحصول على حدود أكثر قوة وعمومية. يتحكم في هذا التساهل معامل يُسمى C: قيم عالية تجعل النموذج صارماً (مع خطر الإفراط)، وقيم منخفضة تجعله متساهلاً (مع خطر التبسيط الزائد).

## متى نتألق SVM؟

تشتهر SVM بأدائها الممتاز في حالات محددة:

البيانات الصغيرة وعالية الأبعاد. لو كان لديك ١٠٠ عينة فقط لكن ١٠٠٠٠ متغير، فالكثير من الخوارزميات تنهار. SVM تتألق هنا، لأنها تعتمد على عدد قليل من متجهات الدعم بدلاً من كامل البيانات.

علم الجينوم. هذا بالضبط ما نواجهه في أبحاثنا: عينات قليلة (لأن جمع البيانات الجينومية مكلف) ومتغيرات كثيرة جداً (الجينات بالآلاف). استخدمت SVM في تصنيف أنماط التعبير الجيني لأنواع مختلفة من السرطان، وكانت نتائجها ممتازة حتى مع بيانات لا تتجاوز ٢٠٠ عينة.

مسائل التصنيف الثنائي. SVM في صورتها الأصلية مصممة لفئتين. يمكن توسيعها لفئات متعددة، لكنها تبقى أفضل في المسائل الثنائية.

## مثال: تصنيف الأورام من بيانات التعبير الجيني

تحليل دراسة لـ ١٥٠ عينة من نسيج صدر، نصفها أورام حميدة ونصفها خبيثة. لكل عينة قياسات لتعبير ٢٠٠٠٠ جين. الهدف: بناء نموذج يميز بين النوعين.

ندرب SVM بنواة RBF. الخوارزمية تجد ٣٠ عينة فقط من أصل ١٥٠ تكون متجهات دعم — أي العينات الحرجة على حدود القرار. بقية العينات لا تؤثر

على الحدود. النموذج النهائي يحقق دقة ٩١٪ على عينات اختبار جديدة. هذا أداء يفوق كثيراً من الخوارزميات الأخرى على بيانات بهذا الحجم الصغير.

## الأخطاء الشائعة عند استخدام SVM

أحد أكبر الأخطاء: نسيان توحيد المقاييس (Standardization). لو كان متغير ما يقاس بالملايين وآخر بالأعشار، ستهيمن المتغيرات الكبيرة على حساب المسافة. دائماً وحد المقاييس قبل التدريب.

خطأ آخر: عدم ضبط معاملي C و  $\gamma$  في نواة RBF. القيم الافتراضية نادراً ما تكون مثلى. استخدم البحث الشبكي (Grid Search) مع التحقق المتقاطع لإيجاد القيم المناسبة.

## الخلاصة

آلة المتجهات الداعمة هي صديقتك الوفية حين تواجه بيانات صغيرة عالية الأبعاد، خاصة في المسائل البيولوجية والطبية. تعلم متى تختار SVM، وأتقن مفهوم الهامش وحيلة النواة، وستجد فيها أداة قوية تُكمل ترسانتك.

## الفصل السادس: خوارزمية الجيران الأقرب (KNN)

﴿الْأَخْلَاءُ يَوْمَئِذٍ بَعْضُهُمْ لِبَعْضٍ عَدُوٌّ إِلَّا الْمُتَّقِينَ﴾

— سورة الزحرف: ٦٧

في هذه الآية تنبيه عميق على أن أخلاءك — جيرانك في الحياة الدنيا — يُحدّدون مصيرك يوم القيامة. إن جالسَ المتقين، صرْتَ منهم؛ وإن جالسَ غيرهم، صرْتَ مثلهم. وهذا بالضبط جوهر خوارزمية الجيران الأقرب: أنت تُعرّف بمن حولك. لا تنظر إلى نفسك معزولاً، بل انظر إلى أقرب جيرانك في فضاء البيانات، وستعرف أيّ فئة تنتمي إليها.

### التشبيه الأعمق: قل لي من جيرانك أقل لك من أنت

تخيّل أنك انتقلت إلى حيّ جديد لا تعرف أحداً فيه. كيف تُكوّن انطباعك عن طبيعة الحيّ؟ تنظر إلى أقرب البيوت. هل سكانها هادئون؟ هل أطفالهم يلعبون في الشوارع؟ هل المنازل مهم بها؟ بناءً على ذلك تستنتج طبيعة منطقتك.

KNN تعمل بنفس المنطق. حين يأتي عينة جديدة لم نرها من قبل، نحسب المسافة بينها وبين كل العينات في بيانات التدريب. ثم نأخذ الجيران الأقرب — قد يكون عددهم ٥ أو ١٠ أو ٢٠ — وننظر إلى أي فئة ينتمون. الفئة الغالبة بينهم هي تنبؤنا.

في الحقيقة، KNN لا نتعلم نموذجاً بالمعنى التقليدي. لا توجد معاملات ننتغير، ولا حدود قرار تُرسم. الخوارزمية تحفظ كل بيانات التدريب، وعند كل تنبؤ تبحث في الجيران. تُسمى هذه الخوارزميات تعلم كسول (Lazy Learning) لأنها تؤجل العمل إلى لحظة التنبؤ.

## اختيار K: لعنة الأرقام الصغيرة والكبيرة

السؤال الجوهرى في هذه الخوارزمية: كم جاراً ننظر إليه؟ هل ثلاثة كافية؟ أم سبعة؟ أم خمسون؟ هذه القيمة تُسمى K، واختيارها فنّ يتطلب توازناً.

K صغير (مثل ١ أو ٣) يجعل النموذج حساساً جداً للضوضاء. يكفي أن يكون أقرب جار شاذاً ليتأثر التنبؤ. الحدود تصبح متعرجة وغير مستقرة.

K كبير (مثل ١٠٠) يُشوِّش الصورة. تصبح كل عينة جديدة "تصوت" بالأغلبية الكبرى للبيانات، فتفقد التفاصيل الدقيقة. الحدود تصبح ناعمة جداً.

القيمة الذكية تقع في الوسط. القاعدة العامة: ابدأ بـ  $K = \sqrt{n}$  حيث  $n$  عدد عينات التدريب، ثم استخدم التحقق المتقاطع لضبطها. وفضل دائماً قيماً فردية في مسائل التصنيف الثنائي لتجنب التعادل.

## كيف نقيس "القرب"؟

كلمة "أقرب" تحتاج إلى تعريف رياضي. أيّ مسافة نستخدم؟ هناك خيارات متعددة، أشهرها:

المسافة الإقليدية (Euclidean). هي مسافة الخط المستقيم بين نقطتين، كما نتعلمها في المدرسة. تستخدم حين تكون البيانات متصلة وذات معنى هندسي طبيعي.

مسافة مانهاتن (Manhattan). تتخيل المسافة وكأنك تسير في شوارع مدينة على شبكة شطرنج: كم خطوة شرقاً، ثم كم خطوة شمالاً. تستخدم حين تكون الأبعاد مستقلة عن بعضها.

مسافة كوسين (Cosine Similarity). تقيس الزاوية بين متجهين بدلاً من المسافة. مفيدة في معالجة النصوص والمستندات.

كلٌّ من هذه المسافات تُعطي نتائج مختلفة. الاختيار يعتمد على طبيعة بياناتك.



صندوق للمتعمقين: المسافة الإقليدية بين نقطتين  $x$  و  $y$  في فضاء  $n$ -بعدي هي:  $d = \sqrt{\sum (x_i - y_i)^2}$  وهذه المسافة تتأثر بشدة بمقاييس المتغيرات. إذا كانت متغيرة تُقاس بالملايين وأخرى بال عشرات، ستُيمن الأولى. لذا التوحيد ضروري.

## نقاط القوة

KNN تتميز بأنها بسيطة جداً في الفهم والتطبيق. لا تحتاج تدريباً معقداً — مجرد حفظ البيانات. كذلك نتعامل بسلاسة مع مسائل التصنيف متعدد الفئات. ولا تفترض أي شكل مسبق للحدود بين الفئات (مثل خط مستقيم في الانحدار أو هامش في SVM)، فهي تتكيف مع البيانات كما هي.

## نقاط الضعف الجدية

لكن لها عيوبها التي يجب أن نعرفها قبل استخدامها.

بطء التنبؤ. لكل عينة جديدة، نحسب الخوارزمية مسافتها إلى كل عينات التدريب. مع ١٠٠٠٠ عينة، هذا ١٠٠٠٠ حساب لكل تنبؤ. على بيانات ضخمة، تصبح بطيئة جداً.

حساسية للميزات غير المهمة. لو كان بعض المتغيرات لا علاقة له بالمشكلة، فإنه سيؤثر على حسابات المسافة ويُسوّج الجيران. دائماً قم بهندسة الميزات وتنقيتها قبل KNN.

لعنة الأبعاد (Curse of Dimensionality). مع زيادة الأبعاد، تصبح كل النقاط متباعدة عن بعضها، ويفقد مفهوم "القرب" معناه. KNN تنهار في الفضاءات عالية الأبعاد دون تقليل الأبعاد أولاً.

حساسية لمقاييس المتغيرات. كما في SVM، يجب توحيد المقاييس قبل التدريب.

## مثال: نظام دعم قرار طبي

تُخيل عيادة تستخدم سجلات ٥٠٠٠ مريض سابق، كل سجل يحوي ١٢ متغيراً سريرياً ومخبرياً، والتشخيص النهائي. عندما يأتي مريض جديد، نريد أن نقترح تشخيصاً مبدئياً للطبيب.

نطبق KNN بـ  $K = 15$ . للمريض الجديد، الخوارزمية تجد أقرب ١٥ مريضاً سبق رؤيتهم. لو كان ١٢ منهم تشخيصهم "التهاب رئوي بكتيري"، نقترح هذا التشخيص. الميزة الجميلة: نستطيع أن نري الطبيب هؤلاء الجيران فعلاً، ونقول له "مريضك يشبه هؤلاء الـ ١٥ مريضاً". هذا ما لا تُعطيه شبكة عصبية مهما كانت دقيقة.

## متى لا تستخدم KNN؟

تجنّب KNN في:

البيانات ذات الأبعاد العالية جداً (مئات أو آلاف المتغيرات) دون تقليل أبعاد.  
البيانات الضخمة جداً (ملايين العينات) لأن التنبؤ سيكون بطيئاً مؤلماً.  
التطبيقات التي تحتاج تنبؤاً في زمن حقيقي.

## الخلاصة

KNN خوارزمية بسيطة وقوية، وغالباً ما تكون نقطة بداية ممتازة لاختبار صعوبة المشكلة. لو لم تستطع KNN أن تُحقّق أداءً مقبولاً، فربما البيانات تحتاج إلى مزيد من المعالجة. تذكر دائماً: أنت تُعرّف بجيرانك، في الحياة وفي البيانات على حدّ سواء.

## الفصل السابع: خوارزميات التعزيز — XGBoost وال Gradient Boosting

---

﴿وَالَّذِينَ جَاهَدُوا فِينَا لَنَهْدِيَنَّهُمْ سُبُلَنَا﴾

— سورة العنكبوت: ٦٩

في هذه الآية معنى عميق: المجاهدة المتدرّجة، الجهد المتراكم، يُؤدي إلى الهداية. لا يهتدي المرء بقفزة واحدة، بل بمحاولات متتالية، كل محاولة تُصحّح ما قبلها وتُكمل النقص. وهذا بالضبط ما تفعله خوارزميات التعزيز (Boosting): تبني نماذج متتابعة، كل نموذج جديد يتعلّم من أخطاء سابقة، حتى يصل النموذج النهائي إلى الإتقان.

### التعزيز مقابل التجميع: نهجان متعاكسان

في الفصل الرابع تعرّفنا على الغابة العشوائية، وهي مثال على فلسفة التجميع (Bagging): نبني أشجاراً مستقلة في الوقت نفسه، ثم نُجمع آراءها. كل شجرة نتعلّم بنفسها، ولا تعرف عن أخواتها شيئاً.

أما التعزيز (Boosting) فهو نهج معاكس تماماً. الأشجار تُبنى بالتتابع، الواحدة بعد الأخرى. كل شجرة جديدة تنظر إلى أخطاء الأشجار السابقة، وتُحاول تصحيحها. النموذج النهائي ليس مجموعة أشجار مستقلة، بل سلسلة أشجار متعاقبة تتراكم لتُعطي تنبؤاً واحداً.

التشبيه: تخيل طالباً يستعد لامتحان. في نهج التجميع، يُحلّ ثلاثة طلاب مستقلون الأسئلة، ثم نأخذ تصويتهم. في نهج التعزيز، طالب أول يُحاول، ثم يأتي طالب ثانٍ ينظر إلى أخطاء الأول ويركّز على ما لم يُجبه. ثم يأتي طالب ثالث ويُكمل ما فات الاثنين. النتيجة النهائية أقوى من أيّ طالب وحده.

## Gradient Boosting: التصحيح بالتدرّج

اسم Gradient Boosting يُشير إلى "التدرّج"، وهو مفهوم رياضي يعني اتجاه أكبر تحسين ممكن. الفكرة الأساسية:

نبدأ بنموذج بسيط جداً (شجرة قرار صغيرة، أو حتى مجرد متوسط). هذا النموذج يُعطي تنبؤات أولية، فيها أخطاء. نحسب لكل عيّنة حجم خطأ التنبؤ.

ثم نبني شجرة جديدة، لكن لا ندرّبها على التنبؤ بالقيمة الأصلية، بل على التنبؤ بالخطأ. هذه الشجرة الثانية تُحاول أن نتعلّم: أين أخطأ النموذج السابق؟ نضيف تنبؤها إلى التنبؤ الأصلي، فيتحسّن.

نُكرّر العملية. شجرة ثالثة نتعلّم أخطاء الاثنين السابقين. شجرة رابعة، خامسة، حتى مئات. في كل خطوة، النموذج يُصحّح نفسه. هذا هو معنى "Gradient": في كل خطوة، نتحرّك في اتجاه يُقلّل الخطأ.

صندوق للتعمّقين: في كل تكرار  $m$ ، النموذج المحدّث هو  $F_m(x)$  =  $F_{(m-1)}(x) + v \cdot h_m(x)$ ، حيث  $h_m$  شجرة جديدة مُدرّبة على البقايا (residuals)، و  $v$  هو معدل التعلّم (learning rate) الذي يتحكّم في حجم خطوة كل شجرة.

## XGBoost: لماذا هيمن على عقد كامل؟

XGBoost — اختصار لـ Extreme Gradient Boosting — تنفيذ ذكي ومحسّن لفكرة Gradient Boosting، ابتكره تيانكي تشن في عام ٢٠١٤. خلال السنوات التالية، فاز في عشرات مسابقات Kaggle، حتى صار يُقال إن "أيّ مسابقة لا يستخدم فيها XGBoost لن تربح".

لماذا هذه الهيمنة؟ XGBoost أضاف عدة تحسينات حاسمة:

التنظيم (Regularization) المضمّن: يُعاقب الخوارزمية الأشجار التي تصبح معقّدة أكثر من اللازم، ما يقلّل الإفراط في التخصيص.

معالجة البيانات الناقصة تلقائياً: لا يحتاج المستخدم إلى تعويض القيم المفقودة يدوياً.

سرعة الحساب: مكتوب بطريقة تستفيد من المعالجة المتوازية والذاكرة الفعالة.

المرونة: يدعم التصنيف، الانحدار، الترتيب، والمسائل متعددة الفئات.

النتيجة: خوارزمية واحدة تتفوق غالباً على الغابة العشوائية، وعلى الشبكات العصبية في كثير من البيانات الجدولية.

## ضبط المعاملات: الفن الحقيقي

XGBoost قوي جداً، لكنه يحتاج ضبطاً دقيقاً ليعطي أفضل أداء. أهم المعاملات:

معدل التعلم (Learning Rate): كم نُضيف من تنبؤ كل شجرة جديدة. قيم صغيرة (٠.٠١ إلى ٠.١) أكثر استقراراً لكنها تحتاج عدداً أكبر من الأشجار.

عدد الأشجار (n\_estimators): عادة بين ١٠٠ و ١٠٠٠. أكثر من اللازم يؤدي للإفراط، أقل من اللازم يؤدي للتقصير.

عمق الشجرة (max\_depth): كم مستوى نتفرّع كل شجرة. عادة بين ٣ و ٨. أعمق يلتقط تفاعلات معقدة لكن مع خطر الإفراط.

معامل التنظيم ( $\lambda, \alpha$ ): يتحكم في حجم العقوبة على التعقيد.

ضبط هذه المعاملات يحتاج تجربة وصبراً. استخدم البحث العشوائي (Random Search) أو البحث الشبكي مع التحقق المتقاطع.

## مثال طبي: التنبؤ بإعادة الإدخال إلى المستشفى

تُخيل دراسة اقترافية على ٢٠٠٠٠ مريض خرجوا من المستشفى. لكل مريض ٤٠ متغيراً سريرياً ومخبرياً وديموغرافياً. الهدف: التنبؤ هل سيعود المريض إلى المستشفى خلال ٣٠ يوماً؟

ندرب نموذج XGBoost. النموذج يبني ٥٠٠ شجرة بالتتابع، كلُّ تُصحَّح أخطاء سابقاتها. النتيجة دقة AUC تصل إلى ٠,٨٢، مقارنة بـ ٠,٧٤ لـ انحدار لوجستي بسيط، و ٠,٧٩ لغابة عشوائية. الفرق قد يبدو صغيراً، لكنه يُترجم إلى عشرات حالات الإدخال المُتجنَّبة شهرياً.

كذلك يُعطينا XGBoost أهمية الميزات. نكتشف أن العمر، عدد الأدوية، عدد المرات السابقة في المستشفى، ومستوى الكرياتينين هي العوامل الأهم. هذه المعرفة تُوجّه التدخل الإكلينيكي.



## متى نختار XGBoost على الغابة العشوائية؟

السؤال الشائع: غابة عشوائية أم XGBoost؟ القاعدة التي اتبعها:

ابدأ بالغابة العشوائية. إذا كانت دقتها كافية، توقف. هي أبسط، أسرع، وأقل احتياجاً للضبط.

إذا كنت تحتاج آخر ٢-٥٪ من الدقة (مثل المسابقات أو التطبيقات الحرجة)، انتقل إلى XGBoost. هي أقوى، لكن تحتاج وقتاً أطول للضبط.

في البيانات الصغيرة ( $> 1000$  عينة)، الغابة العشوائية قد تتفوق لأن XGBoost حساس للإفراط مع البيانات القليلة.

## الأخطاء الشائعة

أكبر خطأ: تشغيل XGBoost بالقيم الافتراضية وتوقع النتيجة المثلثي. القيم الافتراضية نادراً ما تكون مناسبة لبياناتك.

خطأ ثانٍ: تجاهل التوقف المبكر (Early Stopping). هذه ميزة تسمح للنموذج بالتوقف عن إضافة أشجار حين لا يتحسن أدائه على بيانات التحقق. تمنع الإفراط بأناقة.

## الخلاصة

Gradient Boosting و XGBoost هما القمة الحالية في خوارزميات التعلم الآلي الكلاسيكية للبيانات الجدولية. تعلّمها يستحقّ الاستثمار، وستجد نفسك تعتمد عليها في مسائل تتراوح بين التنبؤ المالي والتشخيص الطبي. كما في الآية: من جاهد فينا، لهدّينه سُبُلنا.

## الفصل الثامن: التجميع — K-Means و Hierarchical Clustering

﴿وَجَعَلْنَاكُمْ شُعُوبًا وَقَبَائِلَ لِتَعَارَفُوا﴾

— سورة الحجرات: ١٣

في هذه الآية الكريمة قاعدة سوسيولوجية عظيمة: التنوع البشري ليس فوضى، بل بنية ذات شعوب وقبائل، تجمعهم الاختلافات في مجموعات نتعارف فيما بينها. وهذا بالضبط جوهر التجميع (Clustering): نأخذ بيانات تبدو فوضوية، ونكتشف فيها مجموعات طبيعية من العينات المتشابهة. لا أحد قال لنا مسبقاً ما هي هذه المجموعات؛ نحن نكتشفها من البيانات نفسها.

### التعلم غير الموجه: العثور على البنية بلا إجابات

في كل الفصول السابقة، كانت لدينا بيانات مع إجابات صحيحة: مرضى مع تشخيصاتهم، بيوت مع أسعارها. هذا ما يُسمى التعلم الموجه.

أما التجميع فهو تعلم غير موجه (Unsupervised). لا توجد إجابات. عندنا فقط بيانات: ١٠٠٠٠ خلية مع قياسات تعبيرها الجيني، أو ٥٠٠٠٠ عميل مع

سجلات شرائهم، أو مليون نقطة من بيانات استشعار. نَسأل: هل توجد مجموعات طبيعية فيها؟ هل نستطيع تقسيم البيانات إلى فئات تتشابه داخلياً وتختلف خارجياً؟

التحدي العظيم: ليس لدينا "إجابة صحيحة" نُقارن بها. كيف نعرف أن التجميع نجح؟ هذه أحد الأسئلة الفلسفية في علم البيانات.

## K-Means: قبعات الفرز

أبسط وأشهر خوارزمية تجميع هي K-Means. التشبيه الأقرب: تخيل أنك في مدرسة هاري بوتر، ولديك ١٠٠٠ طالب، وتريد توزيعهم على ٤ بيوت (Houses). كيف تفعل ذلك؟

K-Means تتبع خطوات بسيطة:

أولاً، نختار  $K$  — عدد المجموعات نريدها. لنقل ٤. نضع ٤ نقاط عشوائية في فضاء البيانات، تُسمى المراكز (Centroids).

ثانياً، لكل نقطة في البيانات، نحسب المسافة إلى كل مركز، ونلحقها بأقرب مركز. الآن لكل مركز مجموعة من النقاط.

ثالثاً، نحسب موقع كل مركز جديد كمتوسط النقاط التي تنتمي إليه. المركز يتحرك إلى "قلب" مجموعته.

نُكرّر الخطوتين الثانية والثالثة. النقاط تنتقل بين المجموعات، المراكز تتحرك، حتى لا يتغير شيء — تستقر المجموعات على أماكنها.

النتيجة: تقسيم البيانات إلى  $K$  مجموعات، كل عينة في المجموعة التي مركزها أقرب إليها.

## كيف نختار $K$ ؟ طريقة الكوع

السؤال المؤلم: كم مجموعة في بياناتي؟  $K = 3$  أم  $10$ ؟ الإجابة لا توجد رياضياً، لكن هناك تقنية ذكية تُسمى طريقة الكوع (Elbow Method).

نُجرب عدة قيم لـ  $K$  (مثلاً من 1 إلى 10)، ولكل قيمة نحسب مقياساً يُسمى **Inertia** — مجموع المسافات المربعة بين كل نقطة ومركز مجموعتها. كلما زادت  $K$ ، قلّت Inertia (لأن كل نقطة أقرب إلى مركز).

نرسم Inertia مقابل  $K$ . سترى منحنى ينخفض بسرعة أولاً، ثم يبدأ في الانبطاح. النقطة التي يبدأ فيها الانبطاح تُشبه الكوع. هذه هي  $K$  المثلى. قبلها، إضافة مجموعة تُحسن النتيجة كثيراً. بعدها، التحسين هامشي.

صندوق للمتعمقين:  $K$ -Means تُقلّل دالة الهدف:  $J = \sum_i \sum_{x \in C_i} \|x - \mu_i\|^2$  حيث  $\mu_i$  هو مركز المجموعة  $i$ . الخوارزمية تُضمن التقاء

(Convergence) لكن إلى حدّ أدنى محلي، ليس بالضرورة عالمي. لذا نكرّرها بعدة بدايات عشوائية.

## Hierarchical Clustering: بناء شجرة من التشابه

أسلوب مختلف تماماً عن K-Means هو التجميع الهرمي. هنا لا نختار K مسبقاً. بدلاً من ذلك، نبني شجرة كاملة من التشابهات، تُسمى **Dendrogram**.

في النسخة الصاعدة (Agglomerative)، نبدأ بكل نقطة بمفردها كمجموعة. ثم في كل خطوة، نُدج المجموعتين الأقرب إلى بعضهما. نُكرّر الدمج حتى تصبح كل البيانات في مجموعة واحدة كبيرة. الناتج شجرة تربط كل العينات.

الجمال في هذه الطريقة: بعد بناء الشجرة، نستطيع "قطعها" عند ارتفاع نختاره لنحصل على عدد المجموعات الذي نريد. ارتفاع منخفض = مجموعات كثيرة وصغيرة. ارتفاع عالٍ = مجموعات قليلة وكبيرة. لا نلتزم بـ K مسبقاً.

العيب: بطيئة جداً مع البيانات الضخمة. تحتاج حساب المسافة بين كل زوج من النقاط، ما يؤدي إلى تعقيد  $O(n^2)$  أو أسوأ.

## استخدامات حقيقية

التقسيم الطبقي للمرضى (Patient Stratification). في طب القلب مثلاً، قد نجد أن مرضى قصور القلب ليسوا مجموعة واحدة، بل عدة مجموعات سريرية مختلفة، كل واحدة تستجيب لعلاج مختلف. التجميع يكشف هذه المجموعات الكامنة من البيانات.

أنماط التعبير الجيني. في علم الجينوم، نُجمع جينات بناءً على أنماط تعبيرها عبر العينات. الجينات التي تُعبر معاً غالباً تعمل في المسارات البيولوجية ذاتها. هذه طريقة قديمة لاكتشاف الوظائف الجينية.

تجزئة العملاء (Customer Segmentation). الشركات تُقسّم عملاءها إلى مجموعات بناءً على سلوكهم الشرائي، لتوجيه حملات تسويقية مختلفة لكل مجموعة.

## تجريبي: تجميع أنواع الخلايا في بيانات وحيدة الخلية

في مختبري نُجمع خلايا الكبد بناءً على ملفات تعبيرها الجيني. ندخل ٣٠٠٠٠ خلية إلى الخوارزمية، ونتوقع أن نثر على أنواع خلوية مختلفة — خلايا الكبد الرئيسية (Hepatocytes)، الخلايا المناعية (Macrophages)، الخلايا البطانية (Endothelial)، إلخ.

نستخدم خوارزميات تجميع متقدمة (مثل Leiden، وهي تطوّر لـ K-Means)، ونحصل على ٢٠ مجموعة. ثم نفحص كل مجموعة: ما هي الجينات الأكثر تعبيراً؟ ينطبق ذلك على نوع خلوي معروف. أحياناً نكتشف نوعاً جديداً لم يكن موصوفاً، وهذا اكتشاف علمي حقيقي.

## الأخطاء الشائعة

عدم توحيد المقاييس. مرة أخرى، التجميع يعتمد على المسافات، ولا بدّ من توحيد المقاييس قبل البدء.

إجبار K معين على بيانات لا تستحقّه. أحياناً البيانات لا تنقسم طبيعياً إلى مجموعات. التجميع سيعطي نتيجة، لكنها لا تعكس بنية حقيقية.

التجاهل عن التحقق البيولوجي/المجالي. النتائج الإحصائية يجب أن تُترجم إلى معنى. مجموعة لا يستطيع الخبير المجالي تفسيرها قد تكون مجرد ضوضاء.

## الخلاصة

التجميع أداة قوية للاستكشاف، تُساعدك على فهم بنية بياناتك قبل أن تبني نماذج تنبؤية. ابدأ بـ K-Means، وانتقل إلى التجميع الهرمي حين تحتاج مرونة في عدد المجموعات. وفي العالم الحديث، تُستخدم خوارزميات أكثر تطوراً مثل



DBSCAN و HDBSCAN، لكن المبدأ يبقى واحداً: نحن نبحث عن الشعوب والقبائل في بياناتنا، لتعارف عليها.

## الفصل التاسع: تقليل الأبعاد — PCA و t-SNE و UMAP

﴿وَمِنْ كُلِّ شَيْءٍ خَلَقْنَا زَوْجَيْنِ﴾

— سورة الذاريات: ٤٩

في هذه الآية الكريمة إشارة إلى أن الكثرة الظاهرة في الكون تختزل غالباً إلى أزواج وثنائيات بسيطة. وفي عالم البيانات، نجد ظاهرة مشابهة: قد يكون لديك ٢٠٠٠٠ متغير، لكن في حقيقتها قد تختزل إلى عدد قليل من الأبعاد الجوهرية. تقليل الأبعاد (Dimensionality Reduction) هو فنّ كشف هذه البنية المختزلة، وعرض البيانات المعقدة في فضاء بسيط يُمكن للعقل البشري أن يستوعبه.

### لعنة الأبعاد: لماذا الكثير من الميزات يضرّ؟

قد يبدو غريباً أن نقول "الكثير من البيانات يضرّ النموذج". أليس كل البيانات إضافة قيمة؟ في الحقيقة، لا. عندما يزداد عدد الميزات، تظهر مشاكل عميقة:

أولاً، مشكلة الفضاء الفارغ. في فضاء عشري الأبعاد، كل النقاط تبدو متباعدة عن بعضها. مفهوم "القرب" يفقد معناه. هذا يُحطّم خوارزميات مثل KNN.

ثانياً، زيادة خطر الإفراط (Overfitting). مع كل متغير إضافي، يجد النموذج طرقاً جديدة "لحفظ" البيانات بدلاً من تعميم القاعدة.

ثالثاً، استحالة التصوّر. عقل الإنسان يستوعب ٢ و ٣ أبعاد. ما فوق ذلك، نستحيل أن "نرى" بياناتنا. والرؤية البصرية أداة قوية لاكتشاف الأنماط.

تقليل الأبعاد يُعالج هذه المشاكل: يُلخّص المعلومات في عدد أقل من المتغيرات الجديدة دون فقد جوهري للمعنى.

## PCA: تحليل المكونات الرئيسية

أقدم وأشهر تقنية لتقليل الأبعاد هي تحليل المكونات الرئيسية (Principal Component Analysis). التشبيه الأفضل لها هو تشبيه الظلّ.

تخيّل سحابة نقاط ثلاثية الأبعاد، تمتد بشكل واضح في اتجاه معين. لو سلّطت ضوءاً على هذه السحابة من زاوية معيّنة، يقع ظلّها على الأرض كمسطح ثنائي. لو اخترت الزاوية بحكمة، يحفظ الظلّ معظم المعلومات: انتشار النقاط، تجمعاتها، شكلها العام.

PCA تختار هذه الزاوية بحكمة رياضية. تبحث عن الاتجاهات التي تحوي أعلى تباين (Variance) في البيانات. الاتجاه الأول، أو المكون الرئيسي الأول، هو الخط الذي تنتشر النقاط على طوله أكثر ما يمكن. المكون الثاني عمودي على الأول، ويلتقط ثاني أكبر انتشار. وهكذا.

نأخذ أول مكونين أو ثلاثة، ونرسمهما. النتيجة: تصوّر بياني للبيانات، حتى لو كانت أصلاً في ٢٠٠٠٠ بُعد.

صندوق للمتعمقين: PCA تُحسب رياضياً بإيجاد القيم الذاتية (Eigenvalues) والمتجهات الذاتية (Eigenvectors) لمصفوفة التغاير (Covariance Matrix) للبيانات. المتجهات ذات القيم الذاتية الأكبر تُحدّد المكونات الرئيسية.

PCA خطية: تفترض أن البنية الجوهرية للبيانات يمكن وصفها بمجاور مستقيمة. وهي ممتازة الخطوة أولى، لكنها قد تُفقد البنية المعقدة المنحنية.

## t-SNE: تصوّر بنية محلية معقّدة

في عام ٢٠٠٨، قدّم لورنس فان دير ماتن خوارزمية ثورية اسمها t-SNE (t-distributed Stochastic Neighbor Embedding). الفكرة العبقريّة: بدلاً من حفظ المسافات العالمية، نركّز على حفظ العلاقات المحلية.

السؤال الذي تجيب عليه t-SNE: لو كانت نقطتان قريبتين في الفضاء الأصلي عالي الأبعاد، فلتكونا قريبتين في الفضاء المختزل ثنائي الأبعاد. لا تهتم كثيراً بالمسافات بين نقاط بعيدة جداً.

النتيجة: خرائط جميلة تكشف العناقيد (Clusters) بوضوح مذهل. خلايا من النوع نفسه تتجمّع في "جزيرة" واحدة، وتنفصل عن الأنواع الأخرى. هذا أداة لا غنى عنها في علم الجينوم وحيد الخلية، حيث نريد رؤية البنية النوعية للخلايا.

عيوب t-SNE: بطيئة (تأخذ وقتاً طويلاً مع البيانات الكبيرة)، والمسافات بين العناقيد لا معنى لها (لا تستطيع أن تقول إن مجموعتين بعيدتين على الخريطة بعيدتان بالفعل بيولوجياً).

## UMAP: المعيار الجديد

في عام ٢٠١٨، قدّم ليلاند ماكينز خوارزمية أحدث وأقوى: UMAP (Uniform Manifold Approximation and Projection). UMAP تحلّ كثيراً من مشاكل t-SNE:

أسرع بكثير: تُعالج مليون عيّنة في دقائق، حيث تأخذ t-SNE ساعات.

تحفظ البنية العالمية بشكل أفضل: المسافات بين العناقيد تحمل بعض المعنى.

أكثر استقراراً: تشغيلها مرتين يُعطي نتائج متشابهة، بعكس t-SNE التي قد تُعطي شكلاً مختلفاً في كل مرة.

اليوم، UMAP هو المعيار الفعلي في علم الجينوم وحيد الخلية، وفي تصوّر البيانات عالية الأبعاد عموماً. لو كنت تريد رؤية بياناتك بصرياً، ابدأ ب UMAP.

### ما يمكن وما لا يمكن أن تخبرك به هذه الأدوات

من الأخطاء الشائعة جداً قراءة هذه التصورات بطريقة خاطئة. تذكّر دائماً:

تستطيع أن ترى وجود مجموعات منفصلة، وكثافة النقاط، والشكل العام للبيئة.

لا تستطيع أن تقيس المسافات الفعلية بين نقاط أو مجموعات بدقة. الخريطة تحفظ التشابه المحلي، لا المسافات العامة.

لا تستطيع أن تستخدم التصور للتنبؤ مباشرة. هو أداة استكشاف وفهم، ليس نموذجاً.

## تجربتي: ٣٠٠٠٠ جين في بعدين

في مختبري، نأخذ بيانات الخلية الواحدة بـ ٢٥٠٠٠ جين، ونطبق UMAP. النتيجة: خريطة ثنائية الأبعاد جميلة، فيها كل خلية كنقطة. الخلايا من النوع نفسه تتجمع في عنقيد. لون كل عنقود يكشف نوعاً خلويًا مختلفاً.

في إحدى الدراسات، اكتشفنا عنقوداً صغيراً منعزلاً عن الباقي. تحققنا منه، فإذا هو نوع خلوي نادر يظهر فقط في حالات معينة من السرطان. هذا اكتشاف لم يكن ممكناً دون تقليل الأبعاد. الخريطة جعلتنا نرى ما لم تكن البيانات الخام تكشفه.

## متى تستخدم كل واحدة؟

PCA للاستكشاف الأولي السريع، وحين تكون البنية خطية. أيضاً قبل أيّ خوارزمية حساسة للأبعاد (مثل KNN أو SVM).

t-SNE عندما تريد كشف عناقيد محلية بدقة، وحجم البيانات معقول (أقل من ١٠٠٠٠ عينة).

UMAP للبيانات الكبيرة، وللحصول على توازن بين البنية المحلية والعالمية، وللسرعة. هو خيارك الافتراضي اليوم.

## الخلاصة

تقليل الأبعاد ليس ترفاً، بل ضرورة في عصر البيانات الكبيرة. تعلّم متى تستخدم t-SNE، PCA، أو UMAP، وستفتح لنفسك نافذة على بنى بياناتك لم تكن لتراها بالعين المجردة. كما تختزل الآلة الكثير في زوجين، تختزل هذه الأدوات الفوضى في صورة بسيطة قابلة للفهم.



## الفصل العاشر: التحقق من النموذج وتجنب الإفراط في التخصيص

﴿وَلَا تَقْفُ مَا لَيْسَ لَكَ بِهِ عِلْمٌ﴾

— سورة الإسراء: ٣٦

تنهى الآية الكريمة عن التحدث بما لا علم لك به، وعن البناء على افتراضات لم تُختبر. وفي علم البيانات، أكبر خطيئة هي أن نثق بنموذج لم تختبره على بيانات جديدة. النموذج قد يبدو مثالياً على بيانات تدريبه، ثم يفشل بشكل كارثي في الواقع. هذا الفصل عن كيف نتجنب أن "نقفو ما ليس لنا به علم"، وكيف نقيس ثقتنا في النموذج بطرق صادقة.

### الخطيئة الأولى: تقييم النموذج على بيانات تدريبه

لو سألت طالباً عن أسئلة الامتحان قبل الامتحان، ثم اختبرته على نفس الأسئلة، لحصل على درجة ممتازة. لكن هذا لن يُخبرني شيئاً عن مدى فهمه الحقيقي. لا بد أن أختبره على أسئلة لم يرها من قبل.

في التعلم الآلي، نفس المبدأ. لو دربنا نموذجاً على ١٠٠٠ مريض، ثم اختبرناه على نفس الـ ١٠٠٠، سيعطي دقة عالية. لكن هذه ليست دقته الحقيقية. ربما حفظ النموذج البيانات بدلاً من تعميم القاعدة.

الحل الأساسي: تقسيم البيانات إلى مجموعتين قبل التدريب. عادة ٨٠٪ تدريب و ٢٠٪ اختبار. نُدرّب على الأولى، ونقيس الأداء على الثانية. هذا الأخير هو الأداء الحقيقي.

## التحقق المتقاطع: لجنة المحكّمين الدوّارة

تقسيم ٨٠/٢٠ له مشكلة: قد يكون التقسيم محظوظاً. ربما كانت الـ ٢٠٪ سهلة بالصدفة، فيبدو النموذج أفضل من حقيقته. أو العكس.

الحل: التحقق المتقاطع (Cross-Validation). نقسم البيانات إلى  $K$  أجزاء (عادة ٥ أو ١٠). في كل جولة، نُدرّب على  $K-1$  جزء، ونختبر على الجزء المتبقي. نُكرّر العملية  $K$  مرة، حيث يكون كل جزء بدوره مجموعة الاختبار.

التشبيه: لجنة محاكمة دوّارة، حيث يتناوب كل عضو في دور المحكّم بينما يعمل الآخرون. النتيجة النهائية: متوسط الأداء عبر الجولات الخمس أو العشر. هذه إشارة موثوقة بكثير من تقسيم واحد.

صندوق للمتعمقين: في  $k$ -fold cross-validation، الانحياز ينخفض مع زيادة  $K$ ، لكن التباين يزداد، والوقت الحسابي يتضاعف.  $K=10$  توازن مقبول في معظم الحالات.

## مشكلة جولديلوكس: الإفراط، التقصير، والأمثل

كل نموذج يقع على طيف بين خطأين متطرفين:

التقصير في التخصيص (Underfitting): النموذج بسيط جداً، لا يستطيع التقاط الأنماط في البيانات. مثل خط مستقيم يُحاول وصف منحنى. الأداء سيء على التدريب وعلى الاختبار معاً.

الإفراط في التخصيص (Overfitting): النموذج معقد جداً، يحفظ بيانات التدريب بكل ضوضائها. الأداء ممتاز على التدريب، سيء على الاختبار. هذا الأشد خطورة، لأنه يخدع ظاهرياً.

الحالة الأمثل: نموذج بمستوى تعقيد مناسب يلتقط البنية الحقيقية ويتجاهل الضوضاء. أداء جيد على الاثنين.

كيف نعرف أين نقع؟ نراقب الأداء على التدريب والاختبار معاً مع تغيير تعقيد النموذج. لو كان الأداء على الاثنين سيئاً، نحن في تقصير. لو كان على التدريب ممتازاً وعلى الاختبار سيئاً، نحن في إفراط.

## مفاضلة الانحياز والتباين

هذا مفهوم جوهري في التعلم الآلي يستحق التأمل:

**الانحياز (Bias)** هو مدى انحراف النموذج عن الحقيقة بسبب افتراضاته البسيطة. نموذج خطّي على بيانات منحنية له انحياز عالٍ.

**التباين (Variance)** هو مدى تذبذب توقعات النموذج لو درّب على بيانات مختلفة قليلاً. شجرة قرار عميقة جداً لها تباين عالٍ.

التحدي: تقليل أحدهما يزيد الآخر. نموذج بسيط جداً (انحياز عالٍ، تباين منخفض) يُقصر. نموذج معقد جداً (انحياز منخفض، تباين عالٍ) يُفراط. التوازن المثالي يقع في الوسط.

الغابة العشوائية و XGBoost ينجحان جزئياً لأنهما يوازنان بين هذين بطرق ذكية.

## مقاييس التقييم: لكل سؤال مقياسه

ليست كل المسائل تتطلب نفس المقياس. اختيار المقياس الخاطئ قد يُعطي صورة مضلّة.

**الدقة (Accuracy):** نسبة التنبؤات الصحيحة. مفيدة حين تكون الفئات متوازنة. لكنها مضلّة حين تكون غير متوازنة. لو كان ٩٩٪ من المرضى أصحاء، نموذج يقول دائماً "صحيح" يحقق دقة ٩٩٪، لكنه عديم الفائدة.

**الدقة الإيجابية (Precision):** من بين كل التنبؤات الإيجابية، كم نسبة الصحيحة منها؟ مهمة حين يكون "الإيجابي الكاذب" مكلفاً.

**الاستدعاء (Recall):** من بين كل الحالات الإيجابية الحقيقية، كم نسبة من اكتشفها النموذج؟ مهم حين يكون "السلبي الكاذب" مكلفاً.

**F1:** التوازن بين الدقة والاستدعاء.

**AUC-ROC:** مقياس شامل يقيس قدرة النموذج على التمييز بين الفئتين عبر كل العتبات الممكنة. قيم قريبة من ١ تعني تمييزاً ممتازاً، قيم قريبة من ٠.٥ تعني عشوائية.

في طبّ الأورام مثلاً، الاستدعاء أهم من الدقة الإيجابية: لا نريد أن نفوت مريض سرطان، حتى لو أدى ذلك إلى استدعاء بعض الأصحاء لفحص إضافي.

أما في طبّ الأمراض النفسية حيث الفحص الإضافي مرهق، قد تكون الدقة الإيجابية أهم.

## درس قاسٍ: نموذج بدقة ٩٩٪ كان كارثياً

في مشروع سابق، بنينا نموذجاً للتنبؤ بمرض نادر. الدقة ٩٩٪. اعتقدنا أن الإنجاز عظيم.

لكن المرض كان يُصيب ١٪ فقط من العينة. النموذج تعلّم ببساطة أن يقول دائماً "غير مصاب". الدقة ٩٩٪ كانت خادعة. الاستدعاء كان صفراً. النموذج فاشل تماماً، رغم الرقم الجميل.

هذا الدرس لا يُنسى: اعرّف بياناتك قبل أن تعرف نموذجك. اختر المقياس الذي يعكس فعلاً ما تريد قياسه.

## الأخطاء الشائعة في التحقق

تسرّب البيانات (Data Leakage). حين تسرّب معلومات من الاختبار إلى التدريب. مثلاً، توحيد المقاييس باستخدام البيانات كلها قبل التقسيم. هذا يجعل النموذج يبدو أفضل مما هو.

اختيار الميزات قبل التحقق المتقاطع. لو اخترنا أهم ١٠ متغيرات من كل البيانات، ثم بدأنا التحقق المتقاطع، نكون قد تركنا الاختبار يُؤثر على التدريب. الصحيح: اختيار الميزات داخل كل جولة من التحقق المتقاطع.

التركيز على دقة الاختبار فقط أثناء الضبط. إذا ضبطت معاملاتك بناءً على أداء الاختبار، يصبح الاختبار جزءاً من التدريب فعلياً. الحل: ثلاث مجموعات (تدريب، تحقق، اختبار)، أو التحقق المتقاطع المتداخل.

## الخلاصة

التحقق من النموذج ليس خطوة شكلية، بل هو الجوهر. نموذج لم يُختبر بصرامة هو وعد بلا ضمان. تعلّم استخدام التقسيم، التحقق المتقاطع، والمقاييس المناسبة، فهذه هي الفروق بين عالم بيانات هاوٍ ومحترف. لا تَقِفْ ما ليس لك به علم.

## الفصل الحادي عشر: كيف تختار الخوارزمية المناسبة؟

﴿فَاسْأَلُوا أَهْلَ الذِّكْرِ إِنْ كُنْتُمْ لَا تَعْلَمُونَ﴾

— سورة النحل: ٤٣

أمر الله الناس بالرجوع إلى أهل العلم حين يجهلون. وفي عالم خوارزميات التعلم الآلي، السؤال الأول الذي يحير المبتدئ — وأحياناً المتقدم — هو: أي خوارزمية أستخدم لمشكلتي؟ بعد عشر فصول من تعلم الخوارزميات، يصبح هذا الفصل دليلك العملي للاختيار. سأشاركك خبرة سنوات من التجربة، مُختزلة في إطار قابل للتطبيق.

### نظرية "لا غداء مجاني" (No Free Lunch)

قبل أي شيء، يجب أن تعرف هذا المبدأ المؤسس: لا توجد خوارزمية تتفوق على كل الأخريات في كل المشاكل. الخوارزمية التي تتألق في مسألة قد تفشل في أخرى. هذه ليست لعبة "ابحث عن الأفضل"، بل لعبة "ابحث عن الأنسب".



كل خوارزمية تحمل افتراضات ضمنية. الانحدار الخطي يفترض علاقة خطية. KNN يفترض أن النقاط المتشابهة قريبة في الفضاء. SVM يفترض إمكانية الفصل بهامش. حين تتطابق افتراضات الخوارزمية مع طبيعة بياناتك، تتألق. حين تتعارض، تفشل.

دور عالم البيانات الماهر هو مطابقة الخوارزمية مع المشكلة، لا اعتماد خوارزمية واحدة لكل شيء.

## المحاور الأربعة للاختيار

أعتمد في عملي على أربعة محاور رئيسية لتحديد الخوارزمية:

أولاً: حجم البيانات. كم عينة لدينا؟ مع عينات قليلة ( $> 1000$ )، الخوارزميات البسيطة (انحدار، شجرة قرار، SVM) أفضل لأنها أقل عرضة للإفراط. مع البيانات الكبيرة ( $< 100000$ )، الغابة العشوائية و XGBoost تتألقان. مع البيانات الضخمة جداً (ملايين)، نبدأ التفكير في التعلم العميق.

ثانياً: نوع المتغيرات. هل البيانات أرقام مستمرة فقط؟ فئوية؟ مزيج؟ هل توجد قيم ناقصة كثيرة؟ الغابة العشوائية و XGBoost تتعاملان مع كل الأنواع وتقبلان القيم الناقصة. الانحدار اللوجستي يحتاج أرقاماً منظمّة. الشبكات العصبية تطلّب تحويل المتغيرات الفئوية إلى ترميز ثنائي (One-Hot Encoding).

ثالثاً: متطلّبات التفسيرية. هل سيسأل النموذج "لماذا توقّعت هذا؟" في مجال الطب أو القانون أو الائتمان، التفسيرية إلزامية. الانحدار وشجرة القرار يفوزان هنا. الغابة العشوائية و XGBoost أقل تفسيرية، لكنهما يُعطيان أهمية الميزات. الشبكات العصبية صناديق سوداء.

رابعاً: وقت التدريب والتنبؤ. هل تحتاج تنبؤاً في أجزاء من الثانية؟ الانحدار اللوجستي خفيف وسريع. الغابة العشوائية بطيئة نسبياً عند التنبؤ. XGBoost سريعة بعد التدريب. KNN بطيئة مع البيانات الكبيرة لأنها تحسب التنبؤ من الصفر كل مرة.

## خريطة قرار عملية

دعني أقدم لك خريطة قرار مبسّطة استخدمها فعلياً:

هل تريد التنبؤ بفئة (تصنيف) أم برقم (انحدار)؟

في حالة التصنيف: - بيانات صغيرة (>1000) وعالية الأبعاد: SVM - بيانات متوسطة وأهمية للتفسيرية: انحدار لوجستي أو شجرة قرار - بيانات متوسطة-كبيرة وتريد دقة: غابة عشوائية - بيانات كبيرة وتحتاج أعلى دقة: XGBoost - بيانات ضخمة جداً (ملايين) وغير منظّمة (صور، نصوص): التعلم العميق

في حالة الانحدار: - علاقة خطية واضحة: انحدار خطي - علاقات معقّدة وتفسيرية مهمة: غابة عشوائية - أعلى دقة ممكنة: XGBoost

في حالة استكشاف بنية بلا إجابات (تعلم غير موجه): - اكتشاف عناقيد: K-  
Means أو التجميع الهرمي - تصوّر عالي الأبعاد: UMAP (أو PCA خطوة  
أولى)

## جدول مقارنة شامل

| الخوارزمية        | دقة          | سرعة تدريب            | تفسيرية    | حجم البيانات<br>المثالي |
|-------------------|--------------|-----------------------|------------|-------------------------|
| الانحدار الخطي    | متوسطة عالية | عالية                 | عالية جداً | أيّ حجم                 |
| الانحدار اللوجستي | متوسطة عالية | عالية                 | عالية      | أيّ حجم                 |
| شجرة القرار       | متوسطة عالية | عالية                 | عالية      | متوسط                   |
| الغابة العشوائية  | عالية        | متوسطة                | متوسطة     | متوسط - كبير            |
| SVM               | عالية        | متوسطة - بطيئة        | منخفضة     | صغير - متوسط            |
| KNN               | متوسطة       | عالية (لكن تنبؤ بطيء) | متوسطة     | صغير - متوسط            |
| XGBoost           | عالية جداً   | بطيئة                 | متوسطة     | كبير                    |
| K-Means           | —            | عالية                 | متوسطة     | أيّ حجم                 |

|            |        |                |         |              |
|------------|--------|----------------|---------|--------------|
| الخوارزمية | دقة    | سرعة تدريب     | تفسيرية | حجم البيانات |
| PCA / UMAP | متوسطة | منخفضة أيّ حجم | المثالي |              |

## ورشتي الشخصية: من أين أبدأ؟

في كل مشروع جديد، أتبع هذه الخطوات تقريباً:

أبدأ دائماً بالحدار لوجستي أو شجرة قرار صغيرة. لماذا؟ لأنهما يُعطيني خطّ أساس (Baseline) سريعاً، ويساعداني على فهم بياناتي. لو كانت دقتهما مقبولة، فقد أنتهي من المشروع في يوم.

ثانياً، أنتقل إلى غابة عشوائية. غالباً نتفوق على خطّ الأساس بفارق كبير، وتُعطيني أهمية الميزات لفهم ما يهمّ.

ثالثاً، إذا كانت الدقة مهمة جداً وعندني وقت للضبط، أُجرب XGBoost. غالباً أحصل على ٢-٥٪ دقة إضافية بـ ١٠ دقائق وقت ضبط.

أخيراً، فقط إذا كانت البيانات ضخمة وغير منظّمة (صور، نصوص خام)، أفكّر في التعلم العميق.

## متى تنتقل من الكلاسيكي إلى العميق؟

سؤال مهمّ يطرحه كثيرون: متى أترك هذه الخوارزميات الكلاسيكية وأتعلم الشبكات العصبية العميقة؟

الإجابة: ليس قريباً، إذا كنت تعمل على بيانات جدولية. حتى في عام ٢٠٢٦، ما تزال XGBoost والغابة العشوائية تتفوقان على الشبكات العصبية في معظم البيانات الجدولية. الدراسات الأكاديمية تؤكد ذلك مراراً.

تنتقل إلى التعلم العميق حين: - البيانات صور أو فيديو (الشبكات الالتفافية CNN) - البيانات نصوص أو لغة (ال Transformers) - البيانات تسلسلات زمنية معقدة (RNN، LSTM) - لديك ملايين من العينات وأجهزة قوية

## نصيحة من الميدان

الخطأ الأكبر الذي رأيته في طلابي: قفز مباشر إلى أعقد خوارزمية متاحة، ظناً أنها ستحلّ كل شيء. ينتهون بنماذج معقدة، صعبة الضبط، تتفوق بهامش بسيط على نموذج بسيط كان كافياً.

البساطة ميزة، لا عيب. نموذج بسيط مفهوم وقابل للنشر أفضل بكثير من نموذج معقد لا أحد يفهمه ويصعب صيافته في الإنتاج.

## الخلاصة

اختيار الخوارزمية ليس عشوائياً، ولا يحتاج إلى عبقرية رياضية. يحتاج إلى فهم بياناتك، وفهم متطلبات مشكلتك، ومعرفة نقاط قوة وضعف كل أداة في صندوق أدواتك. تعلّم هذا الفصل جيداً، فهو جسر بين الفصول النظرية والممارسة الفعلية.

## الفصل الثاني عشر: الخاتمة — التعلم الآلي والمستقبل

﴿قُلْ هَلْ يَسْتَوِي الَّذِينَ يَعْلَمُونَ وَالَّذِينَ لَا يَعْلَمُونَ﴾

— سورة الزمر: ٩

في هذه الآية الكريمة سؤال يهزّ الضمير: لا يستوي العالم والجاهل. والعلم في عصرنا لم يعد حكراً على نخبة في برج عاجي، بل صار أداة في يد كل من تعلّم استخدامها. خوارزميات التعلم الآلي التي طُفنا بها في هذا الكتاب هي أدوات معرفية، تُمكنك من فهم ظواهر، اتخاذ قرارات، اكتشاف أنماط، لم يكن ممكناً قبل عقدين. أن نتعلّمها بالعربية وبفهم عميق فهذا، في حدّ ذاته، فعل علمي يستحقّ التقدير.

### خريطة الرحلة: ما الذي قطعناه؟

دعني أوجز معك ما رأيناه:

بدأنا بالانحدار الخطي واللوجستي، لبنتان أساسيتان في كل تعلم آلي. ثم انتقلنا إلى شجرة القرار، أوضح خوارزمية في تفكيرها، وأقربها إلى عقل البشر. الغابة العشوائية و XGBoost أرتانا قوة الجماعة على الفرد. آلة المتجهات الداعمة علّمتنا فكرة الهامش وحيلة النواة. الجيران الأقرب أكّدت أنك تُعرّف بمن حولك.

ثم تركنا التعلم الموجه إلى التعلم غير الموجه: K-Means والتجميع الهرمي يكشفان البنى الكامنة، و PCA, t-SNE, UMAP تختزل بياناتنا المعقدة إلى صور قابلة للفهم.

أخيراً، تعلّمنا في الفصل العاشر كيف نُحقّق من نماذجنا، وفي الحادي عشر كيف نختار الأداة الصحيحة. هذا منهج كامل، ليس مجرد كتاب.

## لماذا الكلاسيكي لم يمت

في عصر يهيمن فيه ChatGPT والشبكات العصبية الضخمة على العناوين، قد يبدو غريباً أن نُكرّس كتاباً للخوارزميات الكلاسيكية. لكن الحقيقة في الميدان أوضح:

معظم البيانات في العالم جدولية: ملفات Excel، قواعد بيانات شركات، سجلات طبية، بيانات مالية. وفي البيانات الجدولية، XGBoost والغلبة العشوائية ما تزالان متفوّقتين على الشبكات العصبية في الدراسات المقارنة.

التفسيرية تزداد قيمتها. كلما زادت قوة النماذج، زادت مخاوف المجتمع من "الصندوق الأسود". قانون الذكاء الاصطناعي الأوروبي (AI Act)، التشريعات الجديدة في أمريكا، كلها تدفع نحو نماذج قابلة للتفسير. الخوارزميات الكلاسيكية في موقع قوة هنا.



كفاءة الموارد. تشغيل XGBoost لا يحتاج بطاقة GPU بألاف الدولارات. هذا ضروري في الدول النامية وللباحثين بميزانيات محدودة.

الكلاسيكي ليس حنيناً، بل اختياراً عقلاً.

## صعود AutoML: حين تختار الآلة الخوارزمية

من التطورات المثيرة في السنوات الأخيرة هو AutoML (التعلم الآلي الآلي): نظم تختار الخوارزمية المناسبة وتضبط معاملاتها أوتوماتيكياً. أدوات مثل Google AutoML و H2O.ai و AutoSklearn تُجرب عشرات النماذج وتُقدم لك الأفضل.

هل هذا يعني أن خبرتك في اختيار الخوارزمية أصبحت بلا فائدة؟ لا، أبداً. AutoML يُسرّع العمل، لكنه لا يفهم سياق مشكلتك. هل التفسيرية مهمة؟ هل البيانات تحوي تحيزاً؟ هل النموذج قابل للنشر في بيئتك الخاصة؟ هذه قرارات بشرية تتطلب فهماً عميقاً، تماماً كما تعلّمناه في هذا الكتاب.

## الأخلاقيات: مسؤوليتنا تجاه ما بنينه

النماذج لا تُصنع في فراغ. كل نموذج يحمل بصمات بيانات تدريبه، التي قد تحوي تحيزات تاريخية. نموذج توظيف مدرب على بيانات شركة كانت تُفضل

الرجال سيتعلّم أن "يُفضّل الرجال". نموذج إقراض مدّرب على بيانات مجتمع تعرض جزء منه لتمييز سيكّرر هذا التمييز.

كعلماء بيانات، علينا مسؤولية:

فحص بياناتنا بحثاً عن تحيّز. هل المجموعات الديموغرافية ممثلة بإنصاف؟ هل البيانات تعكس الحقيقة أم انحيازاً تاريخياً؟

اختبار العدالة في النماذج. هل النموذج يعمل بنفس الدقة عبر فئات السكان؟ هل أخطاؤه موزعة بإنصاف؟

الشفافية مع المستخدمين. حين يتأثر إنسان بقرار نموذج، يحقّ له أن يعرف كيف اتُّخذ.

هذه ليست أمور تقنية فحسب، بل أخلاقية. والقرآن يُدكّرنا: ﴿إِنَّ اللَّهَ يَأْمُرُ بِالْعَدْلِ وَالْإِحْسَانِ﴾.

## نداء للباحث العربي

أكتب هذا الكتاب وأنا في مكتبي بجامعة ميشيغان، أعمل يومياً على بيانات طبية بأدوات تعلّمتها بالإنجليزية، في مراجع كتبها أمريكيون وأوروبيون. لكنني أحلم بجيل من الباحثين العرب يتعلّمون هذه الأدوات بلغتهم الأم، يطبقونها على

بيانات مجتمعاتهم: أمراض السكري في الخليج، السكات القلبية في مصر، السرطان في اليمن، أنماط التعليم في المغرب.

البيانات العربية موجودة، لكنها تحتاج باحثين يفهمون السياق العربي والإسلامي والثقافي. الذكاء الاصطناعي العالمي يُدرّب أساساً على بيانات إنجليزية، ويحمل افتراضاتها. أمتنا تستحق أدواتها الخاصة، نماذجها الخاصة، وعلماءها الخاصين بها. إن قرأت هذا الكتاب وفهمته، فلا تتوقّف عند المعرفة. طبق. اكتب. علّم. أنشر بالعربية. بياناتنا تنتظرنا.

## نصيحة أخيرة للبتي

إذا كنت قد بدأت طريقك للتو في التعلم الآلي، إليك خلاصة الكتاب في فقرة واحدة:

ابدأ بالغة العشوائية. علّم أن تنظّف بياناتك جيداً قبل أن تختار خوارزمية. لا تقع في فخ الخوارزميات المعقّدة قبل أن تُتقن البسيطة. اختبر دائماً على بيانات لم يَرها النموذج. اقرأ نقد بياناتك ونماذجك بقدر ما تقرأ مدحها. علّم النظرية بعد أن تُمارس. اطلب التفسيرية كلما أمكن.

وفوق ذلك كله، تذكّر أن التعلم الآلي أداة، ليس غاية. الغاية أن تحلّ مشكلة حقيقية لإنسان حقيقي. لا تُغرّب بجمال الرياضيات وتنسى الإنسان الذي تخدمه.

## كلمة ختام

في كل خوارزمية درسناها، رأينا بصمة الحكمة الإلهية: العدل في الانحدار، الشورى في الغابة العشوائية، الجوار في KNN، التنوع في التجميع. هذا ليس مصادفة. العلم الحقيقي يُدركنا بحكمة الخالق ويزيدنا تواضعاً.

أتمنى أن تكون قد استفدت من هذا الكتاب. أتمنى أن تشعر الآن بأنك تفهم — لا تحفظ — هذه الخوارزميات. وأتمنى أن تطبقها في خدمة قضية تهتمّ بها، سواء كانت طبية، تعليمية، اقتصادية، أو إنسانية.

والله أعلم.

# القسم الإنجليزي

*English Section*

# Table of Contents

---

|                                                                |   |
|----------------------------------------------------------------|---|
| Chapter 1: What Is Machine Learning?                           | 1 |
| Chapter 2: Linear and Logistic Regression                      | 2 |
| Chapter 3: Decision Trees                                      | 3 |
| Chapter 4: Random Forest                                       | 4 |
| Chapter 5: Support Vector Machines (SVM)                       | 5 |
| Chapter 6: K-Nearest Neighbors (KNN)                           | 6 |
| Chapter 7: Boosting Algorithms — XGBoost and Gradient Boosting | 7 |
| Chapter 8: Clustering — K-Means and Hierarchical Clustering    | 8 |

|                                                            |    |
|------------------------------------------------------------|----|
| Chapter 9: Dimensionality Reduction — PCA, t-SNE, and UMAP | 9  |
| .....                                                      |    |
| Chapter 10: Model Validation and Avoiding Overfitting      | 10 |
| .....                                                      |    |
| Chapter 11: How to Choose the Right Algorithm              | 11 |
| .....                                                      |    |
| Chapter 12: Conclusion — Machine Learning and the Future   | 12 |
| .....                                                      |    |

## Chapter 1: What Is Machine Learning?

---

﴿وَعَلَّمَ آدَمَ الْأَسْمَاءَ كُلَّهَا﴾

*"And He taught Adam the names of all things."* — Surah Al-Baqarah: 31

In this verse there is a profound hint: knowledge begins with naming — with the act of distinguishing things from one another and giving them labels that capture their nature. And this, at its core, is what machine learning does. We do not feed the computer rigid rules; we teach it to *name* — to call an image a cat or a dog, to label a tumor benign or malignant, to separate spam from legitimate email. Machine learning is the art of teaching a machine to recognize patterns, just as Adam was taught the names of all things.



# From Traditional Programming to Pattern Learning

In traditional programming, a developer sits down and writes explicit rules: "if temperature exceeds 38°C, the patient has a fever." Such rules work as long as the problem stays simple and well-defined. But suppose I asked you to write a rule that distinguishes a photograph of a cat from one of a dog. You would hit a wall immediately. Every cat differs from every other in color, size, posture, and lighting. A million rules would not be enough.

Machine learning enters precisely here. Instead of writing rules, we hand the computer thousands of labeled examples — *this is a cat, this is a cat, this is a dog* — and let it derive the rules itself. Patterns are not written; they are discovered. This is a deep shift in how we think about problem-solving: from "*how do I solve this myself?*" to "*how do I show the computer enough examples that it figures it out?*"

# The Three Branches of Machine Learning

Machine learning splits into three main branches, and understanding the difference is your first step.

The first is **supervised learning**. Here we give the computer data along with the correct answers. For example: a thousand patient records with their diagnoses, or a thousand houses with their sale prices. The computer learns the mapping between inputs and outputs and then applies that mapping to new data. Most algorithms in this book — regression, random forest, support vector machines — belong to this branch.

The second is **unsupervised learning**. Here we give the computer data without answers and ask it to find the hidden structure. We say, in effect: "here are ten thousand cells from a cancer biopsy — find the groups that resemble one another." Clustering and dimensionality reduction algorithms — K-Means, PCA, UMAP — are the heroes of this branch.

The third is **reinforcement learning**, the closest to how humans learn. The computer tries actions, receives rewards or penalties, and improves over time. This is how AlphaGo learned to play Go better than any human master. We will not dwell on it in this book, but it is the engine behind many modern applications.

## The Pipeline: From Data to Prediction

Every machine learning project moves through five stages, and understanding them spares you many later mistakes.

It begins with **data**: collecting it, cleaning it, ensuring its quality. Many beginners believe the secret lies in the algorithm, but the truth is that eighty percent of any project's success rests on the data. Bad data will not produce a good model, no matter how clever the algorithm.

Next comes **feature engineering**: transforming raw data into meaningful numbers the model can understand. A birthdate, for example, is useless to a model on its own — but "age in years" is

gold. This stage demands deep domain knowledge, whether medical, financial, or otherwise.

Then comes the **model**: choosing an appropriate algorithm and training it on your data. Then **prediction**: applying the trained model to new, unseen data. Finally, **evaluation**: measuring how well the model performs using clear metrics — the subject of Chapter 10.

## Why Classical Algorithms Still Matter

In an era when deep neural networks dominate the headlines, the reader might wonder: why bother learning random forest or logistic regression at all? My answer, drawn from years of practice, is this: in most real-world problems, classical algorithms are still your best choice.

Deep learning demands millions of examples and powerful hardware. In medical research, you may find yourself working with two hundred patients, period. There, classical algorithms outperform deep networks by a wide margin. They also bring something deep learning struggles to offer: **interpretability**. A

clinician can ask you, "why did your model flag this patient as high-risk?" and you can give a clear answer. That conversation is far harder with a black-box neural network.

## **A View from the Lab**

In my lab at the University of Michigan, we work with single-cell RNA sequencing data to understand diseases like cancer and non-alcoholic fatty liver disease. The data is intricate: a single sample may contain thirty thousand genes measured across tens of thousands of cells. To find rare cell populations or classify cell types, we lean every day on random forest, K-Means, UMAP, and the other algorithms you will meet in this book.

I have personally watched a Random Forest uncover biological signatures we had not suspected, and SVMs classify cancer cells with accuracy that surpasses older manual methods. These are not theoretical tools. They are the daily instruments of research that touches real patients' lives.

# What You Will Learn

In the chapters ahead you will meet a dozen major algorithms, each through its analogy and its real-world example. I will not drown you in mathematics from the start — we will always begin with intuition. When equations appear, they will sit in optional sidebar boxes for the curious reader. The goal is to leave you with deep understanding, not memorized formulas.

***For the curious:** When we say a model "learns," we mean mathematically that it searches for parameter values that minimize a loss function — the gap between its predictions and the true labels. Algorithms differ in the shape of this loss and the way they minimize it, but the underlying principle is shared.*

Remember always: machine learning is not magic. It is the discipline of organizing data well and choosing the right tool. Begin with intuition, and build outward from there.

## Chapter 2: Linear and Logistic Regression

---

﴿إِنَّ اللَّهَ يَأْمُرُ بِالْعَدْلِ وَالْإِحْسَانِ﴾

*"Indeed, Allah commands justice and excellence."* — Surah An-Nahl:

90

Justice, in its deepest sense, is the act of placing every thing where it belongs and finding the balanced relation between cause and effect. Linear regression, at its core, is mathematical justice: we search for the line that does justice to the points — that comes as close as possible to all of them, without leaning unfairly toward any one. Every step of machine learning, in some sense, is an attempt to discover the just relation between variables.

# Linear Regression: When You Want to Predict a Number

Imagine you are selling a house and want to know its fair price. You look at dozens of similar houses in your neighborhood that sold last year. You notice, broadly, that bigger houses sold for more money. If you plotted each house as a point on a sheet of paper — the horizontal axis being area, the vertical axis being price — you would see a cloud of points sloping upward.

Linear regression asks a simple question: *what is the best straight line that runs through this cloud?* A line that, if drawn, would lie as close as possible to all points at once. This line is not just an aesthetic exercise. It is a prediction equation: give me the area of a new house and I will give you its expected price.

The classic image is the **line of best fit**. The model tries to "do justice" to every point — not bending too close to some at the expense of others. How does it measure this fairness? It computes the sum of squared distances between each point and



the line and chooses the line that makes this sum as small as possible. This is what mathematicians call **least squares**.

## What Does "Linear" Mean?

The word *linear* matters. It means the relation between input and output is straight: a one-unit increase in the input corresponds to a fixed increase in the output. If the true relation curves — if, say, price jumps unpredictably above a certain area — then a simple linear regression cannot capture it precisely. Yet it remains a fast, foundational view that shows you the general direction, and it is almost always the right place to start an analysis.

***For the curious:** The basic equation of linear regression is  $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n + \varepsilon$ , where the  $\beta$  values are coefficients learned by the model and  $\varepsilon$  is the unexplained noise.*

## Logistic Regression: When the Question Is Yes or No

But what if you are not predicting a number, but a category? Will this patient develop diabetes — yes or no? Will this dose of medication trigger a response — yes or no? Linear regression cannot help here, because the answer is not an open-ended number but a choice between two options.

Enter **logistic regression**. Its name contains the word *regression*, but it is in fact a classification algorithm. It takes the same inputs and computes a number, but it then passes that number through a magical function called the **sigmoid** that squeezes any value — however large or small — into a number between zero and one. That number is the **probability** of belonging to the positive class.

Picture the sigmoid as a stretched S-shape. At very negative inputs, the output approaches zero. At very positive inputs, it approaches one. At zero, the output is exactly 0.5. This gentle

curve is the algorithm's secret: it converts a straight line (an ordinary regression) into a probabilistic classifier.

## A Medical Example: Predicting

### Diabetic Risk

Suppose, in a hypothetical study of patients carrying diabetes risk factors, we collect eight variables for each: age, body mass index, blood pressure, fasting glucose, number of prior pregnancies, and so on. For each patient we know whether or not they later developed diabetes.

We train a logistic regression model. The model learns that each variable receives a **coefficient** signaling its weight. Fasting glucose, for example, may earn a large positive coefficient: the higher it goes, the higher the risk of diabetes. Blood pressure carries less weight but still contributes. Some variables may earn coefficients near zero, meaning they offer no useful information.

When a new patient arrives, we feed their measurements into the equation, get a number, push it through the sigmoid, and receive a probability — say 0.72 — meaning a 72% predicted chance of disease. If the probability exceeds a chosen threshold (often 0.5), we label the patient "at risk."

## When to Choose Each

The rule is simple. If you are predicting a **continuous number** — price, height, blood concentration — you want linear regression. If you are predicting a **category** — sick versus healthy, will buy versus won't, pass versus fail — you want logistic regression.

Both algorithms are simple, fast, and interpretable. That last virtue is enormous in medicine. When you tell a clinician that "every additional year of age increases the log-odds of disease by 0.04," you are giving them a tool they can debate, audit, and reason about — not a black box.

## Common Mistakes

One of the most frequent errors is applying linear regression to data that is genuinely nonlinear. If the dose-response curve is U-shaped, a straight line will fail at both ends. The remedy is sometimes to transform variables (a logarithm helps often) or to graduate to a more flexible model.

Another pitfall is ignoring **multicollinearity** — when your input variables are themselves strongly correlated. Body weight and BMI, for example. The model becomes unstable and the coefficients hard to interpret. The fix is usually to remove one of the redundant variables or to combine them.

## Closing the Chapter

Linear and logistic regression are not the flashiest algorithms, but they are the foundation on which every modern classification method is built. Master them well, and logistic regression in particular will not disappoint you in many real-world problems. Sometimes the simplest tool is the best one.

## Chapter 3: Decision Trees

---

﴿أَلَمْ تَرَ كَيْفَ ضَرَبَ اللَّهُ مَثَلًا كَلِمَةً طَيِّبَةً كَشَجَرَةٍ طَيِّبَةٍ﴾

*"Have you not seen how Allah sets forth the parable of a goodly word as a goodly tree?" — Surah Ibrahim: 24*

The Quran chooses the tree as a parable because it captures a deep idea: a single root from which many branches spring, each leading to a different fruit. This is exactly what a **decision tree** does in machine learning. It begins with a single question at the root, branches into sub-questions, and arrives at its leaves — the final answers. A goodly tree yields goodly fruit, and a well-trained tree yields accurate predictions.

### The Closest Analogy: A Flowchart That Answers for You

Imagine an internal-medicine clinic where a patient walks in complaining of chest pain. The physician does not order every

test at once. They ask sequential questions. The first: is the patient over sixty? If yes, the second: do they smoke? If yes, the third: is their blood pressure elevated? Based on the answers, the physician arrives at a decision: order an ECG, or prescribe an antacid, or request a troponin test.

This is precisely the shape of a decision tree. A chain of yes/no questions branching outward, with each leaf representing a final decision. The beauty of this analogy is that it is not merely an analogy — it is literally what the algorithm builds.

## The Anatomy: Nodes, Branches, and Leaves

A decision tree has three components.

A **node** is a point holding a question, such as "is age greater than 50?" Branches grow from it.

A **branch** is a path leading either to another node or to a leaf, depending on the answer to the question.

A **leaf** is the path's terminus, holding the final decision — the predicted class or the predicted value.

The first node is called the root. As you descend, the data narrows, until you reach a leaf containing a small group of very similar samples.

## How Does the Tree Choose Its Next Question?

The sharpest question that may strike you is this: who decides what the first question at the root should be, then the second, and so on? The answer: the algorithm itself, through a systematic procedure.

At every node, the algorithm examines every variable and every possible cutoff value, and asks: which question separates the data most cleanly? Which question yields two child groups that are most "pure"?

That word *pure* matters. Suppose we have 100 patients, 50 sick and 50 healthy. That is maximum impurity. But if the model



asks "is glucose above 120?" and the patients split into one group of 40 sick and 5 healthy, and another of 10 sick and 45 healthy, great purity has been achieved. Each group now leans heavily toward a single class.

The algorithm measures this purity using metrics like **entropy** or the **Gini index**. The technical difference between them is small; the shared idea is to pick the question that reduces disorder by the largest possible amount. This is called **information gain**.

*For the curious: The entropy of a binary group with proportions  $p$  and  $(1-p)$  is  $H = -p \log_2(p) - (1-p) \log_2(1-p)$ . It is highest at  $p = 0.5$  (total disorder) and zero at  $p = 0$  or  $1$  (perfect purity).*

## The Overfitting Problem

A greedy tree wants to separate the data perfectly, and it will keep branching until each leaf contains a single sample. This

produces flawless accuracy on the training data, but it is a destructive trap on new data.

The reason is that the tree has *memorized* the training set rather than *learned* from it. It has captured noise and randomness that will never repeat. This is called **overfitting**, one of the deepest problems in all of machine learning.

## Pruning: Curing the Arrogant Tree

The remedy is **pruning**. Just as a gardener trims excess branches so the tree may grow healthier, we trim the branches that add no real predictive value.

Two flavors exist. **Pre-pruning** sets limits before construction begins: do not exceed a certain depth, or do not split a node containing fewer than 20 samples. **Post-pruning** builds the full tree and then removes the branches that contribute to overfitting.

The result is a tree that is less accurate on training data but more accurate on new data — and that is what matters.

## A Clinical Example: The Referral Decision

Picture a primary-care clinic seeing patients who complain of fatigue and abdominal discomfort. The physician wants a tool to help identify who should be referred to a specialist.

We train a decision tree on 2,000 prior records. The tree learns its sequence of questions: is the patient losing weight unexpectedly? If yes, is there blood in the stool? If yes, refer immediately. If no, are liver enzymes elevated? If yes, request additional testing. And so on.

The beauty is that the physician can read the tree and understand the logic of every decision. This is what we call **interpretability**, and it is the decision tree's signature virtue.

## When Not to Use a Decision Tree

Despite its simplicity and charm, the decision tree is not the right answer to every problem. When the data has many

features with complex interactions, a single tree is fragile and falls easily into overfitting. It is also unstable: a small change in the data can completely change the tree's shape.

The solution is not to abandon the idea but to evolve it into the **random forest** — the subject of our next chapter — which combines hundreds of trees into one collective decision that is far stronger.

## Closing

The decision tree is the ideal entry point for understanding classification because it mirrors the way humans actually think: a sequence of questions leading to a verdict. Master it deeply, because it is the building block of many advanced algorithms you will meet, from random forest to XGBoost.

## Chapter 4: Random Forest

---

﴿وَشَاوِرْهُمْ فِي الْأَمْرِ﴾

"And consult them in the matter." — Surah Aal-Imran: 159

Allah commanded His Prophet — peace be upon him — to consult others, even though he received divine revelation, in order to honor the principle of collective wisdom. Minds gathered see angles that a single mind cannot. This is exactly the heart of the **random forest**: we do not ask one decision tree, we ask hundreds, and we combine their voices into a single verdict. It is a mathematical *shura*, council, in its purest form.

### The Wisdom of the Crowd: From One Expert to a Committee

Imagine you need an important medical diagnosis. Would you settle for a single physician's opinion? Most likely you would

seek a second, then a third — especially if the stakes are high. Why? Because each physician carries different experiences, and each may notice what the other missed. When three physicians agree on a diagnosis, your confidence is far greater than your trust in any one alone.

This is the logic of random forest. A single decision tree is intelligent, but it can err. Its mistake on a particular case may happen, while in most cases it gets the answer right. The trick is to consult 100 trees, each trained slightly differently, and then take the collective vote. Individual mistakes cancel each other out, while the correct answer reinforces itself.

## How Do We Build These Trees?

The pressing question is: how do we ensure that each tree in the forest differs from its sisters? If we trained 100 trees on the same data the same way, they would all be identical, and there would be no point in convening a council. The solution is to inject **randomness** in two clever ways.

**Randomness in data (bagging).** For each tree, we draw a random sample of the original data *with replacement*. If we have 1,000 patients, we draw 1,000 records randomly — some patients appear more than once, others not at all. Each tree therefore sees slightly different data.

**Randomness in features (feature sampling).** At every node in the tree, instead of considering every variable, the algorithm picks a random subset — say 5 out of 20. This forces each tree to construct its decisions from a different perspective. One tree may fixate on age and weight; another on lab results.

The result: hundreds of trees, each slightly specialized and each wrong in its own way. When they convene, they converge on the correct answer.

## Voting: How the Verdicts Are Combined

In a classification problem (will this patient develop diabetes?), each tree casts its vote (yes or no), and the forest takes the

**majority.** If 72 trees say "yes" and 28 say "no," the collective verdict is "yes."

In a regression problem (what is the price of this house?), each tree gives a number, and the forest takes the **average**. The simplicity of this aggregation is the algorithm's quiet strength.

*For the curious: Theoretically, combining several "weak but independent" models reduces the variance of the final estimator substantially, provided the trees are not strongly correlated. The less correlated the trees, the better the performance.*

## Why Random Forest Almost Always Wins

In my own work, random forest is often the first thing I try on any new dataset. Why?



It handles high-dimensional data effortlessly. With 5,000 variables, random forest does not flinch. It tolerates missing data gracefully without elaborate preprocessing. It is largely insensitive to whether your features have been standardized to a common scale, unlike algorithms that genuinely require it.

And most importantly: it gives you **feature importance**. After training, the forest can tell you which variables were most useful in its decisions. This is a diagnostic treasure. In medical research we use this importance to discover biological signatures no one had suspected.

## My Experience with Single-Cell Genomics

In my lab at the University of Michigan, we work with single-cell RNA sequencing data on liver cancer. A single sample contains 25,000 genes measured across 30,000 cells. The goal: distinguish cancerous cells from normal ones.

We deployed a random forest with 1,000 trees. The accuracy exceeded 94%. More importantly, the forest produced a ranked list of the 50 genes that contributed most heavily to its decisions. Biological follow-up of those genes revealed that several played a real role in cancer progression that had not been previously emphasized in the literature. A simple tool, an extraordinary outcome.

## **When Random Forest Is Not the Right Choice**

Every algorithm has limits. Random forest is relatively slow at prediction time compared with a simple logistic regression, because it must consult hundreds of trees. If you need predictions in milliseconds across millions of requests, it may be too slow.

It also loses the direct interpretability of a single tree. You cannot "read" a forest of 100 trees the way you read one. There is a trade-off: high accuracy costs you a clear, single chain of reasoning.

## The Out-of-Bag Concept

Among random forest's elegant properties: when each tree is built, roughly one-third of the data is *not* used to train it (because the sample was random with replacement). Those left-out records can be used to evaluate the tree without needing a separate train/test split. This is called **out-of-bag error**, and it is essentially a free estimate of model performance.

## Closing

Random forest is one of the great inventions of practical machine learning. Easy to use, powerful, and flexible. Always begin with it before reaching for more complex algorithms. If its answer is good enough, you do not need to go further. Consultation, as our Quran teaches us, often makes the search for a single oracle unnecessary.

## Chapter 5: Support Vector Machines (SVM)

---

﴿وَجَعَلْنَا بَيْنَهُمْ وَبَيْنَ الْقُرَى الَّتِي بَارَكْنَا فِيهَا قُرًى ظَاهِرَةً﴾

"And We placed between them and the towns which We had blessed [other] visible towns." — Surah Saba: 18

The verse points to the existence of clear partitions between places — partitions that ease passage and define boundaries. The Support Vector Machine, or SVM, rests on a remarkably similar idea: how do we draw the *clearest* divider, the one farthest from every side, so that classification becomes unambiguous and the categories never blur into one another? It is an algorithm that searches for the *widest road* between two classes.

## The Core Analogy: The Widest Road

Imagine a sheet of paper with blue points on one side and red points on the other. You are asked to draw a line that separates them. How many lines are possible? Hundreds. But which is best?

SVM answers: draw the line that lies at the *center of the widest possible road* between the classes. The wider the road, the cleaner the separation. The blue points stay on one curb, the red points stay on the other, and the gap between them is as large as possible. This road is called the **margin**.

The points sitting on the road's edges — closest to the dividing line — are called the **support vectors**. They alone determine the line's position. Remove every other point and nothing changes. This is a deep idea: the algorithm depends on a handful of critical points, which is why it works well even on small datasets.

# What Is a "Hyperplane" Without the Math Panic?

In a two-dimensional world (the paper), the divider is a line. In three dimensions, it is a flat plane. In higher dimensions — say twenty variables — the divider is something hard to picture, called a **hyperplane**.

Do not be intimidated by the word. It is simply the "many-dimensional" version of a straight line. The core idea does not change: we want to separate classes with the largest possible margin in the data's space, no matter how many dimensions that space has.

## The Genius Trick: When Data Cannot Be Separated by a Straight Line

What if the data is not linearly separable at all? Imagine the red points form a circle in the middle and the blue points surround them. No straight line can divide them.

Here enters the **kernel trick**, one of the most beautiful ideas in machine learning. The analogy: imagine a sheet with points that no line can separate. What if you folded the sheet a particular way, or lifted some points upward into three dimensions? In the new space, you might find a flat plane that separates them easily.

The kernel is a mathematical function that maps the data into a higher-dimensional space — sometimes infinitely so — without ever needing to compute the new coordinates explicitly. The common kernels:

**Linear kernel** — no transformation, used when the data is already linearly separable.

**Polynomial kernel** — transforms the data using quadratic, cubic, or higher relations.

**Radial Basis Function (RBF) kernel** — the most popular. It handles complex relations through a centered bell function. It is your default choice when in doubt.

***For the curious:** The RBF kernel is  $K(x, y) = \exp(-\gamma\|x-y\|^2)$ . The parameter  $\gamma$  controls how quickly influence falls off with distance. Small  $\gamma$  produces a smooth boundary; large  $\gamma$  produces a boundary that bends sharply.*

## Hard Margin vs. Soft Margin

In an ideal world a clean gap exists between classes. In the real world, data is messy: blue points sneak among the red, labels are sometimes wrong. Should we force the model to achieve perfect separation, even if it distorts the boundary?

A **hard margin** insists on perfect separation. This is unrealistic in most cases.

A **soft margin** allows a bounded number of mistakes — points crossing the margin or being misclassified — in exchange for a sturdier, more general boundary. The trade-off is governed by a parameter called  $C$ . Large  $C$  makes the model strict (with a risk



of overfitting); small  $C$  makes it lenient (with a risk of underfitting).

## When SVM Shines

SVM is famous for outstanding performance in specific situations.

**Small, high-dimensional data.** With only 100 samples but 10,000 variables, many algorithms collapse. SVM thrives here, because it depends on a handful of support vectors rather than the full dataset.

**Genomics.** This is exactly what we face in our research: few samples (genomic data is expensive to collect) and very many variables (genes number in the tens of thousands). I have used SVM to classify gene-expression patterns across cancer subtypes, with excellent results even on datasets of fewer than 200 samples.

**Binary classification.** SVM in its original form was designed for two classes. It can be extended to many, but it is most natural in binary problems.

## **An Example: Classifying Tumors from Gene-Expression Data**

Picture a study of 150 breast tissue samples, half benign tumors and half malignant. Each sample carries expression measurements for 20,000 genes. The aim is to build a model that distinguishes the two.

We train an SVM with an RBF kernel. The algorithm finds that only 30 of the 150 samples become support vectors — the critical ones living on the decision boundary. The remaining samples do not influence the boundary at all. The final model achieves 91% accuracy on new test samples — a result that outperforms many other algorithms on data of this small size.

# Common Mistakes When Using SVM

One of the largest pitfalls is forgetting to **standardize** features. If one variable is measured in millions and another in tenths, the larger one will dominate the distance calculation. Always rescale variables before training.

Another mistake is failing to tune the  $C$  and  $\gamma$  parameters of the RBF kernel. Default values are rarely optimal. Use grid search with cross-validation to find appropriate values.

## Closing

The Support Vector Machine is your faithful companion when facing small, high-dimensional data — especially in biology and medicine. Learn when to choose SVM, master the concept of the margin and the kernel trick, and you will find it a powerful addition to your toolkit.

## Chapter 6: K-Nearest Neighbors (KNN)

---

﴿الْأَخِلَاءُ يَوْمَئِذٍ بَعْضُهُمْ لِبَعْضٍ عَدُوٌّ إِلَّا الْمُتَّقِينَ﴾

*"Close friends, that Day, will be enemies to each other, except for the righteous."* — Surah Az-Zukhruf: 67

The verse delivers a profound warning: those near you in this life — your companions — shape your fate in the next. Sit with the righteous and you become one of them; sit with others and you become like them. This is the very heart of the **K-Nearest Neighbors** algorithm: you are known by your neighbors. Do not look at yourself in isolation. Look at your closest neighbors in the data space, and you will know which class you belong to.

### The Deeper Analogy: Tell Me Who Your Neighbors Are

Imagine you have moved to a new neighborhood where you know no one. How do you form an impression of the place?

You look at the closest houses. Are the residents quiet? Are children playing in the streets? Are the homes well-maintained? From these clues you infer the character of your area.

KNN works on exactly this logic. When a new sample arrives that we have never seen, we compute the distance between it and every sample in the training data. We then take the **K nearest neighbors** — perhaps 5, or 10, or 20 — and look at which class they belong to. The majority class among them is our prediction.

In truth, KNN does not learn a model in the traditional sense. There are no parameters to fit, no decision boundary to draw. The algorithm simply memorizes the training set, and at every prediction it searches among the neighbors. Such methods are called **lazy learning**, because they postpone the work until the moment of prediction.

# Choosing K: The Curse of Numbers

## Too Small or Too Large

The central question in this algorithm is: how many neighbors should we consult? Three? Seven? Fifty? This value is called  $K$ , and its choice is an art that demands balance.

**Small  $K$**  (such as 1 or 3) makes the model highly sensitive to noise. A single odd nearest neighbor can flip a prediction. Decision boundaries become jagged and unstable.

**Large  $K$**  (such as 100) blurs the picture. Every new sample becomes a vote of the broad majority of the data, and fine detail is lost. Boundaries become overly smooth.

The wise value lies in the middle. A useful starting heuristic is  $K = \sqrt{n}$ , where  $n$  is the number of training samples. Then refine with cross-validation. In binary classification problems, prefer odd values to avoid ties.

## How Do We Measure "Closeness"?

The word *closest* requires a mathematical definition. Which distance do we use? Several options exist; the most common are:

**Euclidean distance.** The straight-line distance between two points, the kind you learned in school. Used when the data is continuous and has a natural geometric meaning.

**Manhattan distance.** Picture distance as walking on a city grid: how many blocks east, then how many north. Used when the dimensions are independent of one another.

**Cosine similarity.** Measures the angle between two vectors rather than the distance between them. Useful in text and document processing.

Each of these choices yields different neighbors and different predictions. Which to use depends on the nature of the data.

***For the curious:** The Euclidean distance between points  $x$  and  $y$  in  $n$ -dimensional space is  $d = \sqrt{\sum (x_i - y_i)^2}$ . This distance is highly sensitive to the scale of variables. If one feature is measured in millions and another in tens, the first will dominate. Standardization is therefore essential.*

## Strengths

KNN is enormously appealing for its **simplicity** of understanding and use. It needs no complex training — only memorization of the data. It also handles **multi-class classification** gracefully and assumes no fixed shape for the decision boundary (no straight line as in regression, no margin as in SVM); it adapts to whatever shape the data presents.

## Serious Weaknesses

But it has flaws you must know before reaching for it.



**Slow prediction.** For every new sample, the algorithm computes its distance to *every* training sample. With 10,000 samples that is 10,000 calculations per prediction. On large datasets it becomes painfully slow.

**Sensitivity to irrelevant features.** If some variables have nothing to do with the problem, they will still contribute to the distance and distort the neighborhood. Always perform feature engineering and selection before KNN.

**The curse of dimensionality.** As dimensions grow, every point becomes roughly equidistant from every other, and the very concept of "near" loses meaning. KNN collapses in high-dimensional spaces unless dimensionality is reduced first.

**Sensitivity to feature scales.** As with SVM, you must standardize variables before training.

## A Clinical Decision-Support Example

Picture a clinic that uses 5,000 historical patient records, each containing 12 clinical and laboratory variables along with a

final diagnosis. When a new patient arrives, we want to suggest a preliminary diagnosis to the physician.

We apply KNN with  $K = 15$ . For the new patient, the algorithm finds the 15 most similar prior patients. If 12 of them carry the diagnosis "bacterial pneumonia," we suggest that diagnosis. The beautiful feature: we can actually show the physician those 15 neighbors and say, "your patient resembles these 15 prior patients." This kind of transparency is something a deep neural network cannot offer, however accurate.

## **When Not to Use KNN**

Avoid KNN with:

Very high-dimensional data (hundreds or thousands of features) without dimensionality reduction. Very large datasets (millions of samples), because prediction becomes painfully slow. Real-time prediction applications.

## Closing

KNN is a simple, surprisingly capable algorithm, and it is often an excellent starting point to gauge the difficulty of a problem. If KNN cannot achieve reasonable performance, the data probably needs more preprocessing. Remember always: you are known by your neighbors, in life and in data alike.

## Chapter 7: Boosting Algorithms — XGBoost and Gradient Boosting

---

﴿وَالَّذِينَ جَاهَدُوا فِينَا لَنَهْدِيَنَّهُمْ سُبُلَنَا﴾

*"And those who strive for Us — We will surely guide them to Our ways."* — Surah Al-Ankabut: 69

The verse carries a deep meaning: gradual striving, accumulated effort, leads to guidance. One does not arrive at understanding in a single leap, but through successive attempts, each correcting what came before and completing what was missing. This is precisely what **boosting algorithms** do: they build models in sequence, each new model learning from the mistakes of its predecessor, until the final model approaches mastery.

# Boosting vs. Bagging: Two Opposite Philosophies

In Chapter 4 we met the random forest, an example of the **bagging** philosophy: we build trees in parallel and combine their votes. Each tree learns independently and knows nothing of its sisters.

**Boosting** is the opposite philosophy. The trees are built **sequentially**, one after another. Each new tree examines the errors of the previous trees and tries to correct them. The final model is not a collection of independent trees, but a chain of consecutive trees that accumulates into a single prediction.

The analogy: imagine a student preparing for an exam. In the bagging approach, three independent students answer the questions and we take the majority vote. In the boosting approach, the first student attempts the exam, then a second student examines the first's mistakes and focuses on what was missed. A third student then completes what the first two could not. The final answer is stronger than any single student's.

# Gradient Boosting: Correction by the Gradient

The name *Gradient Boosting* refers to the *gradient* — a mathematical concept meaning *the direction of greatest possible improvement*. The core idea unfolds like this:

We begin with a very simple model — a small decision tree, or even just an average. This model gives initial predictions, and they contain errors. We compute, for each sample, the size of the prediction error.

Then we build a new tree, but we do not train it to predict the original target. We train it to predict the **error** itself. This second tree tries to learn: where did the previous model go wrong? We add its predictions to the original predictions, and the model improves.

We repeat. A third tree learns the residual errors of the first two. A fourth, a fifth, on through hundreds. At every step, the

model corrects itself. This is what *gradient* means: at each step we move in the direction that reduces the error the most.

***For the curious:*** At iteration  $m$ , the updated model is  $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$ , where  $h_m$  is a new tree trained on the residuals and  $\nu$  is the learning rate that controls each tree's step size.

## XGBoost: Why It Dominated for a Decade

XGBoost — short for *eXtreme Gradient Boosting* — is a clever and highly optimized implementation of gradient boosting, introduced by Tianqi Chen in 2014. In the years that followed, it won dozens of Kaggle competitions, to the point where people said: "*any competition that does not use XGBoost will not be won.*"

Why this dominance? XGBoost added several decisive improvements:

**Built-in regularization** — it penalizes trees that grow more complex than necessary, reducing overfitting.

**Automatic handling of missing data** — the user does not need to fill missing values manually.

**Computational speed** — written to exploit parallel processing and efficient memory.

**Flexibility** — supports classification, regression, ranking, and multi-class problems.

The result is a single algorithm that often outperforms random forest and even neural networks on tabular data.

## Hyperparameter Tuning: The Real Art

XGBoost is enormously powerful, but it demands careful tuning to give its best. The most important parameters:



**Learning rate** — how much we add from each new tree. Small values (0.01 to 0.1) are more stable but require more trees.

**Number of trees (n\_estimators)** — typically between 100 and 1,000. Too many causes overfitting; too few causes underfitting.

**Tree depth (max\_depth)** — how many levels each tree branches to. Typically between 3 and 8. Deeper captures complex interactions at the risk of overfitting.

**Regularization (lambda, alpha)** — controls the strength of the complexity penalty.

Tuning these parameters takes experimentation and patience. Use random search or grid search with cross-validation.

## A Medical Example: Predicting Hospital Readmission

Picture a hypothetical study of 20,000 patients discharged from hospital. For each patient we have 40 clinical, laboratory, and

demographic variables. The goal: predict whether the patient will return to hospital within 30 days.

We train an XGBoost model. It builds 500 trees in sequence, each correcting the previous ones. The result is an AUC of 0.82, compared with 0.74 for simple logistic regression and 0.79 for random forest. The difference may seem small but translates into dozens of avoided admissions per month.

XGBoost also gives us feature importances. We discover that age, number of medications, prior admissions, and creatinine level are the most influential variables. This knowledge guides clinical intervention.

## **When Choose XGBoost over Random Forest?**

The common question: random forest or XGBoost? My rule:

Start with random forest. If its accuracy is sufficient, stop. It is simpler, faster, and demands less tuning.

If you need the last 2–5% of accuracy (in competitions or critical applications), move to XGBoost. It is stronger but requires more time to tune.

On small datasets (fewer than 1,000 samples), random forest may outperform XGBoost, which is sensitive to overfitting on sparse data.

## Common Mistakes

The greatest mistake: running XGBoost with default values and expecting optimal results. The defaults are rarely well-suited to your data.

The second mistake: ignoring **early stopping**. This feature lets the model halt the addition of new trees once performance on a validation set ceases to improve. It elegantly prevents overfitting.

## Closing

Gradient boosting and XGBoost stand at the current peak of classical machine learning for tabular data. Learning them is a worthwhile investment, and you will find yourself relying on them in problems ranging from financial forecasting to medical diagnosis. As the verse says: those who strive will be guided to the path.

## Chapter 8: Clustering — K-Means and Hierarchical Clustering

---

﴿وَجَعَلْنَاكُمْ شُعُوبًا وَقَبَائِلَ لِتَعَارَفُوا﴾

*"And We made you peoples and tribes that you may know one another." — Surah Al-Hujurat: 13*

The verse contains a profound sociological principle: human diversity is not chaos. It is a structure of peoples and tribes, in which differences gather individuals into groups that come to know one another. This is exactly the heart of **clustering**: we take data that looks chaotic and discover within it natural groups of similar samples. No one tells us in advance what those groups are; we uncover them from the data itself.

# Unsupervised Learning: Finding Structure Without Answers

In every previous chapter we had data with correct answers: patients with their diagnoses, houses with their prices. This is **supervised learning**.

Clustering is **unsupervised**. There are no answers. We have only data: 10,000 cells with their gene-expression measurements, or 50,000 customers with their purchase histories, or a million points from a sensor stream. We ask: are there natural groups in here? Can we partition the data into categories that are similar within and different across?

The great challenge: we have no "right answer" against which to compare. How do we know clustering succeeded? This is one of the philosophical questions of data science.

## K-Means: The Sorting Hat

The simplest and most popular clustering algorithm is **K-Means**. The closest analogy: imagine you are at Hogwarts, with 1,000 students, and you wish to assign them to four houses. How do you do it?

K-Means follows simple steps.

First, choose  $K$  — the number of clusters you want. Say four. Place four random points in the data space, called **centroids**.

Second, for every data point, compute its distance to each centroid and assign it to the nearest. Now each centroid has a group of points.

Third, recompute each centroid as the mean of the points assigned to it. The centroid moves to the "heart" of its group.

Repeat steps two and three. Points migrate between groups; centroids drift; the process continues until nothing changes — the clusters settle.

The result: a partition of the data into  $K$  clusters, each sample assigned to the cluster whose centroid is nearest.

## How Do We Choose $K$ ? The Elbow

### Method

The painful question: how many clusters are in my data?  $K = 3$  or 10? There is no mathematical answer, but a clever technique called the **elbow method** comes close.

We try several values of  $K$  (say from 1 to 10), and for each value we compute a metric called **inertia** — the sum of squared distances between every point and the center of its cluster. The larger  $K$  gets, the smaller inertia becomes (since each point lies closer to a center).

We plot inertia against  $K$ . You will see a curve that drops sharply at first, then flattens out. The point where flattening begins resembles an **elbow**. That is the optimal  $K$ . Before it, adding a cluster improves the result substantially. After it, the improvement is marginal.



*For the curious: K-Means minimizes the objective  $J = \sum_i \sum_{x \in C_i} \|x - \mu_i\|^2$ , where  $\mu_i$  is the centroid of cluster  $i$ . The algorithm guarantees convergence — but only to a local minimum, not necessarily the global one. We therefore restart it multiple times from different random initializations.*

## Hierarchical Clustering: Building a Tree of Similarity

A completely different approach is **hierarchical clustering**. Here we do not pre-select  $K$ . Instead, we build a full tree of similarities, called a **dendrogram**.

In the agglomerative version, we begin with every point as its own cluster. Then at every step we merge the two closest clusters. We repeat the merging until all the data has gathered into one large cluster. The output is a tree linking every sample.

The beauty of this approach: after building the tree, we can "cut" it at a height of our choosing to obtain whatever number of clusters we like. A low cut gives many small clusters. A high cut gives few large clusters. We are not pre-committed to  $K$ .

The drawback: very slow on large data. It must compute the distance between every pair of points, leading to  $O(n^2)$  complexity or worse.

## Real-World Uses

**Patient stratification.** In cardiology, for example, we may discover that heart-failure patients are not a single group but several distinct clinical subgroups, each responding to a different treatment. Clustering reveals these latent populations directly from the data.

**Gene-expression patterns.** In genomics we cluster genes by their expression patterns across samples. Genes that vary together often act in the same biological pathway. This is a long-standing way to discover gene function.

**Customer segmentation.** Companies divide their customers into groups based on purchasing behavior, in order to direct different marketing campaigns to each.

## **My Experience: Clustering Cell Types in Single-Cell Data**

In my lab, we cluster liver cells based on their gene-expression profiles. We feed 30,000 cells into the algorithm and expect to recover known cell types — hepatocytes, macrophages, endothelial cells, and so on.

We use advanced clustering algorithms (Leiden is one, an evolution of K-Means) and obtain twenty clusters. We then examine each: which genes are most highly expressed in it? Does it correspond to a known cell type? Sometimes we discover a previously undescribed type, which is a real scientific finding.

## Common Mistakes

**Failing to standardize features.** Once again, clustering depends on distance, and standardization is essential before starting.

**Forcing a particular K onto data that does not deserve it.** Sometimes the data does not divide naturally into groups. Clustering will give you an answer anyway, but it does not reflect real structure.

**Ignoring biological or domain validation.** Statistical results must translate into meaning. A cluster that no domain expert can interpret may be nothing but noise.

## Closing

Clustering is a powerful exploratory tool that helps you understand the structure of your data before building predictive models. Begin with K-Means, and graduate to hierarchical clustering when you need flexibility in the number of clusters. In the modern world, more sophisticated algorithms exist —

DBSCAN, HDBSCAN — but the principle remains one: we are searching for the peoples and tribes within our data, that we may come to know them.

## Chapter 9: Dimensionality Reduction

### — PCA, t-SNE, and UMAP

---

﴿وَمِنْ كُلِّ شَيْءٍ خَلَقْنَا زَوْجَيْنِ﴾

"And of all things We created pairs." — Surah Adh-Dhariyat: 49

The verse hints that the apparent multiplicity of the universe often reduces to simple pairs and dualities. The world of data shows a parallel phenomenon: you may have 20,000 variables, but in truth they may be reducible to a small number of fundamental dimensions. **Dimensionality reduction** is the art of uncovering this reduced structure, and of presenting complex data in a simple space the human mind can absorb.

# The Curse of Dimensionality: Why Too Many Features Hurts

It may sound strange to say "too much data hurts the model." Doesn't every variable add value? In fact, no. As the number of features grows, deep problems arise.

First, the **empty-space problem**. In a ten-dimensional space, every point looks roughly equidistant from every other. The notion of "near" loses meaning. This destroys algorithms like KNN.

Second, **increased risk of overfitting**. With every additional variable, the model finds new ways to *memorize* the data instead of generalizing.

Third, **the impossibility of visualization**. The human mind grasps two and three dimensions. Beyond that we cannot "see" our data — and visual inspection is a powerful pattern-finding tool.

Dimensionality reduction addresses all three: it summarizes information into fewer new variables without losing the essential meaning.

## PCA: Principal Component Analysis

The oldest and most popular technique is **Principal Component Analysis**. The best analogy is the **shadow analogy**.

Imagine a three-dimensional cloud of points stretched out clearly in some direction. If you shine a light onto this cloud from a particular angle, its shadow falls onto the ground as a two-dimensional pattern. If you choose the angle wisely, the shadow preserves most of the information: the spread of points, their groupings, their general shape.

PCA chooses that angle by mathematical wisdom. It searches for the **directions in which the data has the highest variance**. The first direction, the **first principal component**, is the line along which the points spread out the most. The



second component is perpendicular to the first and captures the second-largest spread. And so on.

We take the first two or three components and plot them. The result: a visual representation of the data, even if it originally lived in 20,000 dimensions.

***For the curious:** PCA is computed mathematically by finding the eigenvalues and eigenvectors of the covariance matrix of the data. The eigenvectors with the largest eigenvalues define the principal components.*

PCA is **linear**: it assumes the underlying structure of the data can be described by straight axes. It is excellent as a first step but may miss complex curved structure.

## **t-SNE: Visualizing Local Structure**

In 2008, Laurens van der Maaten introduced a revolutionary algorithm called **t-SNE** (t-distributed Stochastic Neighbor

Embedding). Its genius idea: instead of preserving global distances, focus on **preserving local relationships**.

The question t-SNE answers: if two points are close in the original high-dimensional space, let them remain close in the reduced two-dimensional space. Do not worry too much about distances between points that are very far apart.

The result: beautiful maps that reveal **clusters** with astonishing clarity. Cells of the same type gather into a single "island," distinct from other types. This is an indispensable tool in single-cell genomics, where we want to see the categorical structure of cells.

t-SNE's drawbacks: it is slow (taking hours on large datasets), and the distances between clusters are not meaningful (you cannot say two distant clusters on the map are biologically distant).

# UMAP: The New Standard

In 2018, Leland McInnes introduced a newer and more powerful algorithm: **UMAP** (Uniform Manifold Approximation and Projection). UMAP solves many of t-SNE's problems.

**Far faster:** it processes a million samples in minutes, where t-SNE takes hours.

**Better preservation of global structure:** distances between clusters carry some meaning.

**More stable:** running it twice gives similar results, unlike t-SNE which may produce a different shape every time.

Today, UMAP is the de facto standard in single-cell genomics and in high-dimensional data visualization generally. If you want to see your data visually, start with UMAP.

# What These Tools Can and Cannot Tell You

A common mistake is reading these visualizations the wrong way. Always remember:

You **can** see the existence of separate groups, the density of points, and the general shape of the structure.

You **cannot** measure actual distances between points or clusters precisely. The map preserves local similarity, not global distance.

You **cannot** use the visualization to predict directly. It is an exploration and understanding tool, not a model.

## My Experience: 30,000 Cells in Two Dimensions

In my lab we take single-cell data with 25,000 genes and apply UMAP. The result: a beautiful two-dimensional map with each

cell as a point. Cells of the same type gather into clusters. The color of each cluster reveals a different cell type.

In one study we discovered a small cluster isolated from the rest. We investigated it, and it turned out to be a rare cell type that appears only in particular cancer subtypes. This finding was not possible without dimensionality reduction. The map made us see what the raw data did not reveal.

## When to Use Each

**PCA** for fast initial exploration, and when the structure is approximately linear. Also as a preprocessing step before any algorithm sensitive to dimensionality (KNN, SVM).

**t-SNE** when you want to reveal local clusters with high clarity, and the dataset is moderate in size (fewer than 10,000 samples).

**UMAP** for large data, for a balance between local and global structure, and for speed. It is your default choice today.

## Closing

Dimensionality reduction is not a luxury but a necessity in the era of big data. Learn when to use PCA, t-SNE, or UMAP, and you will open for yourself a window onto the structure of your data that you would not have seen with the naked eye. As the verse reduces multiplicity to pairs, these tools reduce chaos to a comprehensible image.

## Chapter 10: Model Validation and Avoiding Overfitting

---

﴿وَلَا تَقْفُ مَا لَيْسَ لَكَ بِهِ عِلْمٌ﴾

*"And do not pursue that of which you have no knowledge."* — Surah Al-Isra: 36

The verse forbids speaking of what you do not know and building on untested assumptions. In data science, the gravest sin is to trust a model you have not validated on new data. The model may look perfect on its training data and then fail catastrophically in reality. This chapter is about how to avoid "pursuing what you do not know," and how to measure your confidence in a model honestly.

# The First Sin: Evaluating a Model on Its Own Training Data

If I gave a student the exam questions before the exam, then tested them on the same questions, they would score brilliantly. But that score would tell me nothing about their real understanding. I must test them on questions they have never seen.

In machine learning, the same principle holds. If we train a model on 1,000 patients and then test it on those same 1,000 patients, the accuracy will be high. But that is not the model's true accuracy. It may have memorized the data rather than generalizing the rule.

The fundamental remedy: **split the data** into two groups before training. Typically 80% for training and 20% for testing. We train on the first and measure performance on the second. The latter is the real performance.



# Cross-Validation: The Rotating Jury

The 80/20 split has a weakness: it might be lucky. The 20% may have happened to be easy, making the model look better than it is — or the reverse.

The remedy is **cross-validation**. We split the data into  $K$  parts (commonly 5 or 10). In each round, we train on  $K-1$  parts and test on the remaining one. We repeat  $K$  times, with each part taking its turn as the test set.

The analogy: a rotating jury, where each member takes their turn as judge while the others sit. The final result is the average performance across all  $K$  rounds. This is a far more reliable signal than any single split.

***For the curious:** In  $k$ -fold cross-validation, bias decreases as  $K$  grows but variance increases, and computational time multiplies.  $K = 10$  is a balanced choice in most circumstances.*

# The Goldilocks Problem: Underfitting, Overfitting, and the Sweet Spot

Every model lies on a spectrum between two extremes.

**Underfitting:** the model is too simple to capture the patterns in the data. Like a straight line trying to describe a curve. Performance is poor on both training and test sets.

**Overfitting:** the model is too complex; it memorizes the training data with all its noise. Performance is excellent on training, poor on test. This is the more dangerous failure, because it deceives at first glance.

**The sweet spot:** a model with appropriate complexity that captures real structure and ignores noise. Good performance on both.

How do we know where we sit? We monitor performance on training and test as we vary model complexity. If performance is poor on both, we are underfitting. If excellent on training but poor on test, we are overfitting.

# The Bias-Variance Trade-off

This is a foundational concept in machine learning, worth meditating on.

**Bias** is how far the model deviates from the truth because of its simplifying assumptions. A linear model on curved data has high bias.

**Variance** is how much the model's predictions fluctuate when trained on slightly different data. A very deep decision tree has high variance.

The challenge: reducing one tends to increase the other. A very simple model (high bias, low variance) underfits. A very complex model (low bias, high variance) overfits. The optimal balance lies in the middle.

Random forest and XGBoost succeed in part because they balance bias and variance through clever mechanisms.

# Evaluation Metrics: Each Question Has Its Own Yardstick

Not every problem demands the same metric. Choosing the wrong metric can produce a misleading picture.

**Accuracy:** the fraction of correct predictions. Useful when classes are balanced. Misleading when they are not. If 99% of patients are healthy, a model that always says "healthy" achieves 99% accuracy — and is useless.

**Precision:** of all positive predictions, what fraction are correct? Important when false positives are costly.

**Recall:** of all true positive cases, what fraction did the model catch? Important when false negatives are costly.

**F1:** the balance between precision and recall.

**AUC-ROC:** a comprehensive metric that measures the model's ability to distinguish the two classes across all possible

thresholds. Values near 1 indicate excellent discrimination; values near 0.5 indicate random.

In oncology, recall matters more than precision: we do not want to miss a cancer patient, even if it means recalling some healthy people for additional testing. In a screening context where additional tests are burdensome, precision may matter more.

## **A Hard Lesson: A Model with 99% Accuracy That Was Disastrous**

In a past project we built a model to predict a rare disease. Accuracy was 99%. We thought we had achieved something great.

But the disease affected only 1% of the sample. The model had learned to always say "not affected." The 99% accuracy was deceptive. Recall was zero. The model was a complete failure, despite the beautiful number.

The lesson is unforgettable: **know your data before you know your model**. Choose a metric that actually reflects what you want to measure.

## Common Mistakes in Validation

**Data leakage.** When information from the test set leaks into training. For example, standardizing using all the data before splitting. This makes the model look better than it is.

**Feature selection before cross-validation.** If we choose the top 10 variables from the entire dataset and then begin cross-validation, we have allowed the test set to influence training. The right approach: select features inside each cross-validation fold.

**Tuning on the test set.** If you tune your hyperparameters using test-set performance, the test set becomes part of training in effect. The remedy: three sets (train, validation, test), or nested cross-validation.

## Closing

Model validation is not a formality but the very heart of the discipline. A model that has not been rigorously tested is a promise without a guarantee. Master the use of splits, cross-validation, and appropriate metrics — these are the differences between an amateur data scientist and a professional one. Do not pursue that of which you have no knowledge.

# Chapter 11: How to Choose the Right Algorithm

---

﴿فَاسْأَلُوا أَهْلَ الذِّكْرِ إِنْ كُنْتُمْ لَا تَعْلَمُونَ﴾

"So ask the people of knowledge if you do not know." — Surah An-Nahl: 43

Allah commands people to consult those who have knowledge when they do not know. In the world of machine learning algorithms, the first question that confuses the beginner — and sometimes the experienced — is: *which algorithm should I use for my problem?* After ten chapters of learning algorithms, this chapter becomes your practical guide for choosing among them. I will share with you years of experience, condensed into an applicable framework.



# The "No Free Lunch" Theorem

Before anything else, you must know this founding principle:

**no algorithm outperforms every other on every problem.**

The algorithm that shines on one task may fail on another. This is not a "find the best" game — it is a "find the most appropriate" game.

Every algorithm carries implicit assumptions. Linear regression assumes a linear relationship. KNN assumes that similar points lie close in the feature space. SVM assumes a separable margin. When the algorithm's assumptions match the nature of your data, it shines. When they conflict, it fails.

The skilled data scientist's role is to **match the algorithm to the problem**, not to use a single favorite for every task.

## The Four Axes of Choice

In my own work I rely on four main axes to determine the algorithm.

**First: data size.** How many samples do we have? With few samples (under 1,000), simple algorithms (regression, decision tree, SVM) are best because they are less prone to overfitting. With large data (over 100,000), random forest and XGBoost shine. With very large data (millions), we begin thinking about deep learning.

**Second: variable types.** Is the data continuous numbers only? Categorical? A mix? Are there many missing values? Random forest and XGBoost handle every type and tolerate missing values. Logistic regression needs structured numbers. Neural networks require categorical variables to be one-hot encoded.

**Third: interpretability requirements.** Will the model be asked "why did you predict this?" In medicine, law, or credit decisions, interpretability is mandatory. Regression and decision trees win here. Random forest and XGBoost are less interpretable, but they give feature importance. Neural networks are black boxes.

**Fourth: training and prediction time.** Do you need predictions in milliseconds? Logistic regression is light and fast.

Random forest is relatively slow at prediction time. XGBoost is fast after training. KNN is slow on large data because it computes from scratch every time.

## A Practical Decision Map

Let me give you the simplified decision map I actually use.

**Are you predicting a category (classification) or a number (regression)?**

**For classification:** - Small data (<1,000) and high-dimensional: SVM - Medium data, interpretability matters: logistic regression or decision tree - Medium-to-large data, you want accuracy: random forest - Large data, top accuracy needed: XGBoost - Very large data (millions) and unstructured (images, text): deep learning

**For regression:** - Clear linear relationship: linear regression - Complex relationships and interpretability matters: random forest - Highest possible accuracy: XGBoost

**For exploring structure without labels (unsupervised):** -  
Discovering clusters: K-Means or hierarchical clustering - High-dimensional visualization: UMAP (or PCA as a first step)

## A Comprehensive Comparison Table

| Algorithm           | Accuracy | Training speed         | Interpretability | Ideal data size |
|---------------------|----------|------------------------|------------------|-----------------|
| Linear regression   | Medium   | Fast                   | Very high        | Any size        |
| Logistic regression | Medium   | Fast                   | High             | Any size        |
| Decision tree       | Medium   | Fast                   | High             | Medium          |
| Random forest       | High     | Medium                 | Medium           | Medium-large    |
| SVM                 | High     | Medium-slow            | Low              | Small-medium    |
| KNN                 | Medium   | Fast (slow prediction) | Medium           | Small-medium    |

| Algorithm     | Accuracy  | Training speed | Interpretability | Ideal data size |
|---------------|-----------|----------------|------------------|-----------------|
| XGBoost       | Very high | Slow           | Medium           | Large           |
| K-Means       | —         | Fast           | Medium           | Any size        |
| PCA /<br>UMAP | —         | Medium         | Low              | Any size        |

## My Personal Workflow: Where I Begin

For every new project I follow approximately these steps.

I always begin with a **logistic regression** or a **small decision tree**. Why? They give me a quick **baseline** and help me understand my data. If their accuracy is acceptable, I may finish the project in a day.

Second, I move to a **random forest**. It often outperforms the baseline by a wide margin, and gives me feature importance to understand what matters.

Third, if accuracy is critical and I have time to tune, I try **XGBoost**. Often I gain 2–5% additional accuracy at the cost of tuning time.

Finally, only if the data is large and unstructured (images, raw text), do I think about **deep learning**.

## When Do You Move from Classical to Deep Learning?

A common question: when do I leave these classical algorithms and learn deep neural networks?

The answer: not soon, if you work on tabular data. Even in 2026, XGBoost and random forest still outperform neural networks on most tabular data. Academic studies confirm this repeatedly.

Move to deep learning when: - The data is images or video (CNNs) - The data is text or language (Transformers) - The data is complex time series (RNNs, LSTMs) - You have millions of samples and powerful hardware

## A Lesson from the Field

The biggest mistake I have seen in my students: jumping immediately to the most complex algorithm available, expecting it to solve everything. They end up with complex models, hard to tune, that outperform a simple model by a hair when the simple model would have sufficed.

Simplicity is a virtue, not a defect. A simple, understandable, deployable model is far better than a complex one that no one understands and that is hard to maintain in production.

## Closing

Algorithm choice is not random, and it does not require mathematical genius. It requires understanding your data, understanding your problem's requirements, and knowing the strengths and weaknesses of each tool in your toolbox. Master this chapter well — it is the bridge between the theoretical chapters and actual practice.

## Chapter 12: Conclusion — Machine Learning and the Future

---

﴿قُلْ هَلْ يَسْتَوِي الَّذِينَ يَعْلَمُونَ وَالَّذِينَ لَا يَعْلَمُونَ﴾

"Say: Are those who know equal to those who do not know?" —  
Surah Az-Zumar: 9

The verse poses a question that stirs the conscience: the knower and the unknower are not equal. In our era, knowledge is no longer the preserve of an elite in an ivory tower. It has become a tool in the hand of anyone who learns to use it. The machine learning algorithms we have toured in this book are tools of knowledge that allow you to understand phenomena, make decisions, and discover patterns that were not possible two decades ago. Learning them in your native language with deep understanding is, in itself, a worthy scientific act.



# The Map of the Journey: What We Have Covered

Let me summarize with you what we have seen.

We began with **linear and logistic regression**, two foundation stones of all machine learning. Then we moved to the **decision tree**, the most transparent algorithm in its reasoning and the closest to the way humans think. **Random forest** and **XGBoost** showed us the power of the collective over the individual. **Support Vector Machines** taught us the idea of the margin and the kernel trick. **K-Nearest Neighbors** confirmed that you are known by your neighbors.

Then we left supervised for unsupervised learning: **K-Means** and hierarchical clustering reveal latent structure, and **PCA**, **t-SNE**, **UMAP** reduce our complex data to images we can comprehend.

Finally, in Chapter 10 we learned how to validate our models, and in Chapter 11 how to choose the right tool. This is a complete curriculum, not merely a book.

## Why Classical Has Not Died

In an era when ChatGPT and giant neural networks dominate the headlines, it may seem strange to devote a book to classical algorithms. But the truth in the field is clearer.

**Most data in the world is tabular:** Excel files, corporate databases, medical records, financial data. On tabular data, XGBoost and random forest still outperform neural networks in comparative studies.

**Interpretability grows in value.** As models grow stronger, society's concern about "black boxes" grows. The European AI Act, new American regulations — all push toward interpretable models. Classical algorithms stand on strong ground here.

**Resource efficiency.** Running XGBoost does not require a GPU costing thousands of dollars. This is essential in developing nations and for researchers on limited budgets.

Classical is not nostalgia. It is rational choice.

## The Rise of AutoML: When the Machine Picks the Algorithm

Among the exciting developments of recent years is **AutoML** — systems that select the appropriate algorithm and tune its parameters automatically. Tools like Google AutoML, H2O.ai, and AutoSklearn try dozens of models and present you with the best.

Does this mean your expertise in algorithm selection has become useless? No, not at all. AutoML accelerates work, but it does not understand the context of your problem. Does interpretability matter? Does the data contain bias? Is the model deployable in your particular environment? These are human

decisions that demand deep understanding — exactly what we have learned in this book.

## Ethics: Our Responsibility Toward What We Build

Models are not made in a vacuum. Every model carries the fingerprints of its training data, which may contain historical biases. A hiring model trained on data from a company that historically favored men will learn to "favor men." A lending model trained on data from a society where part of the population suffered discrimination will perpetuate that discrimination.

As data scientists we bear responsibilities.

**Audit our data for bias.** Are demographic groups represented fairly? Does the data reflect truth or historical bias?

**Test models for fairness.** Does the model perform equally well across population groups? Are its errors distributed equitably?

**Be transparent with users.** When a person is affected by a model's decision, they deserve to know how it was made.

These are not merely technical matters but ethical ones. The Quran reminds us: *"Indeed, Allah commands justice and excellence."*

## **A Call to the Arabic-Speaking Researcher**

I write this book from my office at the University of Michigan, working daily on medical data with tools I learned in English, from references written by Americans and Europeans. But I dream of a generation of Arab researchers who learn these tools in their native tongue and apply them to the data of their own communities: diabetes in the Gulf, heart disease in Egypt, cancer in Yemen, education patterns in Morocco.

Arabic-language data exists, but it needs researchers who understand the Arab and Islamic and cultural context. Global AI is trained primarily on English data and carries its assumptions.

Our nation deserves its own tools, its own models, and its own scientists.

If you have read this book and understood it, do not stop at the knowledge. Apply. Write. Teach. Publish in Arabic. Our data awaits us.

## **A Final Word for the Beginner**

If you have just begun your path in machine learning, here is the essence of this book in one paragraph:

Begin with random forest. Learn to clean your data well before you choose an algorithm. Do not fall into the trap of complex algorithms before mastering simple ones. Always test on data the model has never seen. Read criticism of your data and models as much as you read praise. Learn theory after you practice. Demand interpretability whenever possible.

Above all, remember that machine learning is a tool, not a goal. The goal is to solve a real problem for a real person. Do not be

dazzled by the beauty of mathematics and forget the human you serve.

## Closing Word

In every algorithm we studied, we saw the imprint of divine wisdom: justice in regression, consultation in random forest, neighborhood in KNN, diversity in clustering. This is no coincidence. True science reminds us of the wisdom of the Creator and increases our humility.

I hope you have benefited from this book. I hope you now feel that you *understand* — not merely memorize — these algorithms. And I hope you will apply them in service of a cause you care about, whether medical, educational, economic, or humanitarian.

*And Allah knows best.*

## عن الكتاب

---

دليل شامل ومبسط لأهم خوارزميات التعلم الآلي الكلاسيكي من الانحدار الخطي إلى الغابة العشوائية وآلة المتجهات الداعمة وما بعدها. يشرح المؤلف كل خوارزمية بالتشبيه والمثال قبل الرياضيات مستندا إلى تجربته في أبحاث علم الجينوم والبيانات الطبية في جامعة ميشيغان.



## عن المؤلف

---

الدكتور فضل محمد الأكوع باحث في البيولوجيا الحاسوبية والمعلوماتية الحيوية والجينومات المكانية بجامعة ميشيغان في آن آربر. تتمحور أبحاثه حول تطبيق الذكاء الاصطناعي في دراسة الأمراض المزمنة، ومن أبرز أعماله المنشورة دراسات على مثبطات ناقل الصوديوم-غلوكوز 2 (SGLT2) والتغيرات الأنوية الكلوية المنشورة في مجلة JCI عام 2023. إلى جانب مسيرته العلمية، يكتب د. فضل باللغتين العربية والإنجليزية في حقول متعددة تشمل الدراسات القرآنية، تاريخ اليمن والسياسة، الذكاء الاصطناعي والتقنية، الأسرة والتربية، ودليل المهاجرين في المهجر.

## مؤلفات المؤلف الأخرى

---

من آلة الكاش إلى نظام البيع الذكي (2026)

كل ما تحتاجه هو الانتباه (2026)

قراءة شرائح الكلى المرضية دليل عملي لتفسير صبغة H&E (2026)

الدليل العملي لمهنة ميكانيكي السيارات في أمريكا (2026)

أفضل المدن للعيش في الولايات المتحدة الأمريكية (2026)

عالم القهوة رحلة من المزرعة إلى الفنجان (2026)

دليل الاستثمار في سوق الأسهم التاريخ والمستقبل وأسرار الثروة (2026)

المثلية الجنسية قراءة في التاريخ والحجج والأديان (2026)

بايثون للذكاء الاصطناعي: من الصفر إلى البناء (2026)

معالجة اللغة العربية مع نماذج الذكاء الاصطناعي: العمل بلغتك الأم (2026)

---

البناء مع كلود – دليل عملي لواجهة برمجة Anthropic (2026)

---

بناء معرض أعمالك في مجال الذكاء الاصطناعي: 30 مشروعاً حقيقياً (2026)

---

هندسة الأوامر – فن التحوار مع الذكاء الاصطناعي (2026)

---

الذكاء الاصطناعي لصنّاع المحتوى - دليل التصميم والصوت والفيديو (2026)

---

الأتمة بدون كود - دليل بناء سير عمل ذكي (2026)

---

العملاء الأذكياء و بروتوكول MCP - بناء أنظمة ذكاء اصطناعي مستقلة (2026)

---

التوليد المعزز بالاسترجاع - دليل عملي لبناء أنظمة RAG (2026)

---

العمل الحر بالذكاء الاصطناعي – دليل المستقل اليمني والعربي (2026)

---

نوت بوك إل إم – الدليل الشامل (2026)

---

القانون الجميل الكبير - تشريح إصلاح ترامب الضريبي وتأثيره على أمريكا (2026)

---

التقارب السني الشيعي (2026)

الذكاء الاصطناعي ومستقبل المهن – أيها يندثر وأيها يصعد (2026)

التأثيرات الاستثمارية في أمريكا – دليل المقارنة والاختيار (2026)

البن في ملفات إبستين (2026)

الرئيس إبراهيم الحمدى – حياته ومشروعه الوطني (2026)

الشخصيات العامة التي فقدت وظائفها بسبب موقفها من غزة (2026)

الصوفية – دراسة شاملة (2026)

الطعام والصحة بين العلم الحديث والحديث النبوي (2026)

النظام الضريبي الأمريكي – دليل شامل (2026)

دليل شراء اللحم الحلال في ميشيجان وإنديانا (2026)

سلسلة التوريد – دليل شامل (2026)

ميكانيكا الكم – دراسة شاملة (2026)

أدوات الذكاء الاصطناعي – دليل شامل (2026)

Mythos في مواجهة نماذج Anthropic (2026)

مهارات كلود - كيف يجعلك استخدامها خبيراً في كل شيء (2026)

البوذية والهندوسية - النشأة والانتشار والمقارنة (2026)

الذنوب والمعاصي والأخلاق في الإسلام - مقارنة مع الأديان  
الأخرى (2026)

الذكاء الاصطناعي وثورة اكتشاف الأدوية (2026)

حرب غزة - كيف فضحت المجتمع الدولي (2026)

مجازر ضد المسلمين عبر العصور (2026)

العلاقات اليمنية السعودية - تاريخ وتحولات عبر الأزمان (2026)

كيف تؤسس شركتك في أمريكا (2026)

هندسة البيانات - الدليل الشامل (2026)

جماعة الدعوة والتبليغ - دراسة شاملة (2026)

أجيال على المحك - الإسلام في المهجر بين الحفاظ والانسلاخ (2026)

المسلمون في الغرب — الإنجازات والتحديات والاندماج (2026)

الربا في الإسلام والبنوك الإسلامية (2026)

القرآن والإنجيل — دراسة مقارنة في مفاهيم السلام والقتال  
والتسامح (2026)

نهاية الكون في القرآن الكريم (2026)

مشاكل اليمن عبر الأزمان (2026)

سر التقارب الأمريكي الإسرائيلي — من يقود من؟ (2026)

هل انتهت الوحدة اليمنية؟ (2026)

الرؤساء اليمنيون في مذكرات ويكيليكس (2026)

أسرار قوة التحالف الأمريكي الإسرائيلي (2026)

فن التربية — كيف تصنع قادة من أبنائك (2026)

تأثير النماذج اللغوية الكبيرة على البحث العلمي والتعليم الجامعي (2026)

علامات الساعة (2026)

التخطيط للتقاعد (2026)

العملات الرقمية — دليل شامل للمبتدئين (2026)

وادي السيليكون — تاريخ وابتكار وريادة علمية (2026)

أشهر المتنزهات الوطنية في أمريكا (2026)

دليل شراء وبيع السيارات المستعملة في أمريكا (2026)

كلود كود — الدليل الشامل (2026)

دليل المهاجر اليمني الجديد إلى أمريكا (2026)

الحقوق الزوجية والحياة الأسرية للمسلمين في الغرب (2026)

مستقبل المعلوماتية الحيوية والبيولوجيا الحاسوبية في عصر الذكاء الاصطناعي (2026)

تصنيف الآيات في القرآن الكريم — دراسة تأملية في البنية الموضوعية (2026)

كيف تعمل النماذج اللغوية الكبيرة — دليل مبسّط (2026)

سرقة الجامعات الأمريكية (2026)

عتيقة — حكاية جدتي من جبال اليمن (2026)

---

الخلافة الإسلامية: من السقيفة إلى سقوط غرناطة (2026)

---

السلفية — دراسة في المفهوم والنشأة والتيارات المعاصرة (2026)

---

النقد المشروع وحدود معاداة السامية (2026)

---

كيف تنجح أكاديمياً في الجامعات الأمريكية (2026)

---

الزواج الناجح — دليل عملي بمنظور إسلامي (2026)

---

الإعجاز العلمي في القرآن — من منظور باحث في الجينوميات (2026)

---

أمريكا — صعود القوة وبوادر الأفول (2026)

---



## About this Book

---

A comprehensive and intuitive guide to classical machine learning algorithms from linear regression to SVM, random forest, XGBoost, clustering, and dimensionality reduction. The author, a computational biologist at the University of Michigan, explains every algorithm through analogy and real biomedical examples before any equations appear.

## About the Author

---

Dr. Fadhl Mohammed Alakwaa is a researcher in computational biology, bioinformatics, and spatial genomics at the University of Michigan in Ann Arbor. His research focuses on applying AI to the study of chronic diseases; among his notable publications are studies on SGLT2 inhibitors and kidney tubular changes published in JCI in 2023. Alongside his scientific career, Dr. Alakwaa writes in both Arabic and English across fields including Quranic studies, Yemeni history and politics, AI and technology, family and parenting, and guides for diaspora life.